ingenieur
wissenschaften
htw saar

Hochschule für
Technik und Wirtschaft
des Saarlandes
University of
Applied Sciences

Bachelor-Thesis

zur Erlangung des akademischen Grades
Bachelor of Science (B. Sc.)
an der Hochschule für Technik und Wirtschaft des Saarlandes
im Studiengang Praktische Informatik
der Fakultät für Ingenieurwissenschaften

Evaluierung lokaler KI-Modelle auf Android für BCI-Anwendungen

vorgelegt von
Ahmed Chaabo

betreut und begutachtet von
Prof. Dr. rer. nat. Dr. rer. med. habil. Daniel J. Strauss
Tobias Grün M.Sc.

Saarbrücken, Tag. Monat Jahr

Selbständigkeitserklärung

Ich versichere, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Ich erkläre hiermit weiterhin, dass die vorgelegte Arbeit zuvor weder von mir noch von einer anderen Person an dieser oder einer anderen Hochschule eingereicht wurde.

Darüber hinaus ist mir bekannt, dass die Unrichtigkeit dieser Erklärung eine Benotung der Arbeit mit der Note „nicht ausreichend“ zur Folge hat und einen Ausschluss von der Erbringung weiterer Prüfungsleistungen zur Folge haben kann.

Saarbrücken, Tag. Monat Jahr

Ahmed Chaabo

KI-Verzeichnis

Im Folgenden werden die im Rahmen dieser Arbeit eingesetzten KI-basierten Hilfsmittel sowie Art und Umfang ihrer Nutzung offengelegt. Die inhaltliche Verantwortung für die gesamte Arbeit liegt beim Verfasser. KI-Werkzeuge wurden ausschließlich unterstützend eingesetzt; alle damit erzeugten oder überarbeiteten Inhalte wurden geprüft, angepasst und verantwortet.

Nr.	KI-Werkzeug	Art der Nutzung	Betroffene Abschnitte
1	Claude (Anthropic)	Rechtschreib- und Grammatikkorrektur sowie verbesserte Formulierungsvorschläge.	Gesamte Arbeit
2	Claude (Anthropic)	Unterstützung bei der Konzeption von Entwürfen für Kapitel.	Gesamte Arbeit
3	Claude (Anthropic)	Unterstützung bei der Konzeption oder Erstellung von Entwürfen für Abbildungen, Listings, Tabellen oder Formeln.	Abb. 4.x, 5.x, 6.x, A.x; Tab. 6.x, B.1; Lis. 5.x; For. 2.2, 4.x
4	Claude (Anthropic)	Unterstützung bei Programmcode durch Vorschläge (Pseudocode), Korrekturen und Optimierungen.	Einzelne im Quellcode gekennzeichnete Methoden
5	Perplexity (Perplexity AI)	Unterstützung bei Recherche und Einordnung von Quellen sowie von Literatureinträgen.	Literaturverzeichnis
6	DeepL (DeepL SE)	Übersetzung von Textpassagen aus dem Englischen ins Deutsche.	Alle englischsprachigen Quellen (Literaturverzeichnis)

Zusammenfassung

In der vorliegenden Arbeit wird die Eignung lokaler Sprachmodelle für mobile Brain-Computer-Interface-Anwendungen (BCI) untersucht. Untersuchungsgegenstand sind drei lokal auf einem Android-Gerät ausgeführte Sprachmodelle, die in drei festgelegten Anwendungsszenarien anhand von Latenz, Ressourcenverbrauch, Stabilität und Ausgabequalität verglichen werden. In der bestehenden Applikation wurden Daten der Elektroenzephalographie (EEG) von einem tragbaren mobilen BCI-Gerät (Unicorn Hybrid Black) erfasst und anschließend visualisiert und/oder gespeichert. Solche Signale sind komplex und erfordern Fachwissen, um sie einwandfrei deuten zu können. Lokale Sprachmodelle könnten hier unterstützen, indem sie die aufbereiteten Messwerte in verständliche Sprache überführen. Das grundlegende Problem ist, dass solche Modelle oft auf ressourcenbeschränkten Geräten laufen und man daher die Tauglichkeit nicht garantieren kann. Daher ergibt sich die Forschungsfrage, inwieweit sich lokal auf Android-Geräten ausgeführte Sprachmodelle für die Verarbeitung von EEG-Daten im Rahmen mobiler Brain-Computer-Interface-Anwendungen eignen.

Aus diesem Grund wurde die Anwendung um drei Kernkomponenten erweitert. Darunter fallen die Merkmalsextraktion, bei der die rohen EEG-Daten in eine für das Modell verständliche Form gebracht werden, die Anbindung lokaler Sprachmodelle (Gemma 3 1B, Gemma 2 2B, Qwen 2.5 1.5B) und die Implementierung einer Benutzeroberfläche. Zusätzlich wurde ein Benchmark implementiert, der für das automatisierte Messen der Latenz, des Ressourcenverbrauchs und der Stabilität der Modelle zuständig ist. Dabei ist die bestehende Anwendung selbst nicht Gegenstand der Bewertung, sondern dient als Messumgebung, in der die Modelle verglichen werden.

Die Ergebnisse zeigen, dass die Verarbeitung der EEG-Daten durch alle drei getesteten Modelle stabil ist. Die Modellgröße bestimmt zwar die Zeit bis zum ersten Token (TTFT), Geschwindigkeit und Speicher, aber nicht automatisch Qualität oder Gesamtverbrauch. Keines der getesteten Modelle ist gleichzeitig in allen Bereichen überlegen. Die passende Modellwahl hängt schließlich davon ab, welcher Einsatzzweck verfolgt wird. Während Qwen 2.5 1.5B nicht überzeugend auftritt, sind die beiden Gemma-Modelle gut geeignet. Für schnelle Antworten auf ressourcenärmeren Geräten ist Gemma 3 1B die beste Wahl. Wenn Zeit keine Rolle spielt und die Antwort sowieso im Hintergrund generiert wird, ist Gemma 2 2B die bessere Wahl, da sie die beste Ausgabequalität liefert.

Zusammenfassend lässt sich sagen, dass sich solche Modelle unter den untersuchten Bedingungen für die Verarbeitung von EEG-Daten eignen. Sie sind eine gute Ergänzung für die bestehende Anwendung und lassen viel Raum für weiterführende Untersuchungen.

If the human brain were so simple that we could understand it,
we would be so simple that we couldn't.

— Emerson M. Pugh

Danksagung

Ich bedanke mich bei meinen Betreuern, Prof. Dr. rer. nat. Dr. rer. med. habil. Daniel J. Strauss und Tobias Grün M.Sc., für die Unterstützung und die Möglichkeit, diese Arbeit im Rahmen des Projekts UnicornART zu schreiben.

Zudem gilt mein besonderer Dank meiner Familie und meinen Freunden, die mich während der Zeit im Studium motiviert und mir den Rücken gestärkt haben.

Inhaltsverzeichnis

1 Einleitung **1**
- 1.1 Motivation . . . 1
- 1.2 Zielsetzung . . . 2
- 1.3 Aufbau der Arbeit . . . 3

2 Grundlagen **5**
- 2.1 Elektroenzephalographie (EEG) . . . 5
- 2.2 Frequenzbänder . . . 5
- 2.3 Frequenzanalyse . . . 6
  - 2.3.1 Fourier-Transformation . . . 6
  - 2.3.2 Fensterung und spektrale Leckage . . . 7
  - 2.3.3 Welch-Methode . . . 7
- 2.4 Lokale Sprachmodelle auf mobilen Geräten . . . 7
  - 2.4.1 Large Language Models . . . 7
  - 2.4.2 On-Device-Inferenz und Quantisierung . . . 8
  - 2.4.3 Modell und Laufzeitumgebung . . . 9
  - 2.4.4 Systemintegrierte und frei deploybare Modelle . . . 9
- 2.5 Brain-Computer-Interface (BCI) . . . 10

3 Stand der Technik **11**
- 3.1 On-Device-Inferenz-Lösungen . . . 11
  - 3.1.1 Systemintegrierte Lösung . . . 11
  - 3.1.2 Frei deploybare Lösung . . . 11
- 3.2 Die Gemma-Modellfamilie . . . 12
- 3.3 Die Qwen-Modellfamilie . . . 13
- 3.4 Bestehende Anwendung . . . 13
  - 3.4.1 Unicorn Hybrid Black . . . 13
  - 3.4.2 Datenübertragung und Anbindung . . . 14
  - 3.4.3 Funktionsumfang der Anwendung . . . 14
  - 3.4.4 Beitrag dieser Arbeit . . . 15

4 Methodik **17**
- 4.1 Merkmalsextraktion . . . 17
  - 4.1.1 Segmentierung . . . 19
  - 4.1.2 Spektrale Merkmale . . . 20
  - 4.1.3 Zeitstatistische Merkmale . . . 20
  - 4.1.4 Aufbereitung für das Sprachmodell . . . 21
- 4.2 Modellauswahl . . . 22
- 4.3 Anwendungsszenarien . . . 22
- 4.4 Messprotokoll (Benchmark) . . . 23
  - 4.4.1 Metriken . . . 25
  - 4.4.2 Messbedingungen . . . 26

5 Implementierung **29**
5.1 Architektur . . . 29
5.2 Implementierung der Merkmalsextraktion . . . 32
5.2.1 Vorbereitung . . . 32
5.2.2 Validierung . . . 33
5.2.3 Segmentierung . . . 33
5.2.4 Spektrale Merkmale (Welch-Methode) . . . 34
5.2.5 Zuordnung zu den Frequenzbändern . . . 36
5.2.6 Statistische Merkmale . . . 36
5.2.7 Ergebnis . . . 36
5.2.8 Aufbereitung für das Sprachmodell . . . 37
5.3 Integration des Sprachmodells . . . 38
5.3.1 Modellauswahl und Pfade . . . 38
5.3.2 Laden und Konfiguration . . . 38
5.3.3 Streamende Ausgabe . . . 40
5.3.4 Zählen der Token . . . 40
5.3.5 Freigabe . . . 40
5.4 Prompt-Design und iterative Optimierung . . . 41
5.5 Benutzeroberfläche . . . 42
5.5.1 Analyse-Dialog . . . 43
5.5.2 Ergebnis-Ansicht . . . 44
5.5.3 Benchmark-Dialog . . . 47
5.6 Benchmark . . . 48
5.6.1 Ablauf . . . 49
5.6.2 Messung je Konfiguration . . . 49
5.6.3 Erfassung der Ressourcen . . . 50
5.6.4 Protokollierung . . . 51

6 Evaluation **53**
6.1 Durchführung der Messungen . . . 53
6.2 Latenz . . . 53
6.3 Ressourcenverbrauch . . . 56
6.4 Stabilität . . . 57
6.5 Ausgabequalität . . . 57
6.6 Zusammenfassende Gegenüberstellung . . . 59

7 Diskussion **61**
7.1 Einordnung der Ergebnisse . . . 61
7.2 Einschränkungen . . . 62
7.3 Handlungsempfehlungen . . . 64
7.3.1 Allgemeine Empfehlungen . . . 64
7.3.2 Modellempfehlung . . . 64

8 Fazit und Ausblick **65**
8.1 Fazit . . . 65
8.2 Ausblick . . . 65

Literatur **67**

Abbildungsverzeichnis **71**

Tabellenverzeichnis **71**

Listings **72**

Abkürzungsverzeichnis **73**

A Aufbereitete Feature-Texte der Evaluation **77**

B Rohdaten der Benchmark-Messungen **79**

1 Einleitung

Gehirn-Computer-Schnittstellen und lokal ausführbare künstliche Intelligenz haben sich in den letzten Jahren stark weiterentwickelt. Die Kombination beider Technologien eröffnet neue Möglichkeiten, unter anderem den Einsatz lokaler Sprachmodelle zur Verarbeitung von Elektroenzephalographie (EEG)-Daten auf einem mobilen Android-Gerät. Die vorliegende Arbeit untersucht, wie diese Kombination in der Praxis funktioniert. Dieses Kapitel gibt einen Überblick über die Motivation, Zielsetzung und den Aufbau der Arbeit.

1.1 Motivation

Das EEG ist ein etabliertes Verfahren zur nicht-invasiven Aufzeichnung der elektrischen Aktivität des Gehirns [4]. Eine Gehirn-Computer-Schnittstelle, ein sogenanntes Brain-Computer-Interface (BCI), zielt anhand dieser Grundlage darauf ab, kognitive Zustände in Echtzeit zu identifizieren, um eine Steuerung und Kommunikation ohne jeglichen Einsatz von Muskeln zu ermöglichen [4]. Vor allem körperlich stark beeinträchtigte Menschen profitieren von BCI-Systemen. Gelähmten Personen ist es unter anderem so möglich, BCI-basierte Buchstabiersysteme zu nutzen, die einen wichtigen Kommunikationskanal darstellen [4].

Traditionelle EEG-Laborhardware, wie sie oft stationär verwendet wird, ist groß und unpraktisch für den Gebrauch außerhalb einer Laborumgebung. Demgegenüber erlauben moderne, kabellose und am Kopf getragene EEG-Geräte (wie das Unicorn Hybrid Black) inzwischen auch Messungen außerhalb des Labors [4, 9]. Dennoch ist das Deuten der aufgezeichneten Signale komplex und benötigt oft Fachwissen. Gleichzeitig haben große Sprachmodelle, sogenannte Large Language Models (LLMs), die Verarbeitung natürlicher Sprache stark geprägt [38]. Für die vorliegende Arbeit ist die lokale Ausführung dabei nicht nur eine technische Möglichkeit, sondern eine Voraussetzung. Da das Unicorn Hybrid Black auch in der Lehre eingesetzt wird, wie in Vorlesungen zu Gehirn-Computer-Schnittstellen, ist weder eine verlässliche Internetverbindung noch ein Zugang zu einem Cloud-Dienst garantiert. Hinzu kommt, dass EEG-Aufzeichnungen personenbezogene Daten sind, die das Gerät nicht verlassen sollen. Eine lokale Verarbeitung kommt ohne Netzanbindung aus und hält die Daten auf dem Gerät [38].

Durch die Kombination lokaler Sprachmodelle und der aufbereiteten EEG-Daten ist es möglich, verständliche und textuelle Beschreibungen zu erzeugen. Der Betrieb solcher Modelle auf Mobilgeräten wird jedoch durch erhebliche rechentechnische Limitierungen sowie durch einen stark begrenzten Arbeits- und Energiespeicher eingeschränkt [38]. Inwieweit sich lokale Sprachmodelle unter diesen ressourcenbeschränkten Bedingungen tatsächlich für die Verarbeitung von EEG-Daten eignen, ist bislang offen.

1.2 Zielsetzung

Ziel dieser Arbeit ist die systematische Untersuchung, ob und inwieweit sich lokal auf Android-Geräten ausgeführte Sprachmodelle für die Verarbeitung von EEG-Daten im Rahmen mobiler Brain-Computer-Interface-Anwendungen eignen.

Damit stellt sich folgende Forschungsfrage:

> *Inwieweit eignen sich lokal auf Android-Geräten ausgeführte Sprachmodelle für die Verarbeitung von EEG-Daten im Rahmen mobiler Brain-Computer-Interface-Anwendungen?*

Um diese Frage zu beantworten, wird eine bestehende Android-EEG-Anwendung um lokale Sprachmodelle erweitert, die dann aufbereitete EEG-Merkmale in verständliche, textuelle Beschreibungen übersetzen. Dabei werden drei verschiedene Anwendungsszenarien sowie drei verschiedene Sprachmodelle mit unterschiedlicher Größe und unterschiedlichem Hersteller betrachtet. So ist es möglich, sowohl den Einfluss der Modellgröße als auch des Herstellers auf die jeweiligen Szenarien zu untersuchen.

Die Eignung wird schlussendlich anhand verschiedener Kriterien bewertet. Dazu zählen Latenz, Ressourcenverbrauch, Stabilität sowie die Qualität der ausgegebenen Texte. Darüber hinaus werden praktische Einschränkungen wie die Geräteabhängigkeit systemintegrierter Modelle dokumentiert. Das Ergebnis dieser Arbeit liefert einen systematischen Vergleich, aus dem sich Handlungsempfehlungen für den Einsatz lokaler Sprachmodelle in mobilen BCI-Anwendungen ableiten lassen.

Der Fokus liegt dabei nicht auf dem Training der Modelle, sondern auf ihrer Integration, Leistungsfähigkeit und ihren Grenzen im mobilen Einsatz.

1.3 Aufbau der Arbeit

Der Aufbau der Arbeit gliedert sich in acht Kapitel:

- **Kapitel 2** vermittelt alles an notwendigem Wissen, das für das Verständnis der Arbeit erforderlich ist.
- **Kapitel 3** beschreibt den derzeitigen Stand der Technik.
- **Kapitel 4** erläutert die Methodik, also das Vorgehen, mit dem die Forschungsfrage beantwortet werden soll.
- **Kapitel 5** stellt dar, wie die konkrete Implementierung der Anwendung erfolgt ist.
- **Kapitel 6** präsentiert die Ergebnisse der Evaluation.
- **Kapitel 7** diskutiert diese Ergebnisse sowie die praktischen Einschränkungen.
- **Kapitel 8** schließt die Arbeit mit einem Fazit und einem Ausblick auf mögliche Weiterentwicklungen ab.

2 Grundlagen

In diesem Kapitel werden die Grundlagen, die für das Verständnis der Arbeit benötigt werden, vermittelt. Dabei werden folgende zentrale Konzepte vorgestellt, auf denen die weiteren Kapitel aufbauen.

2.1 Elektroenzephalographie (EEG)

Das EEG ist ein Verfahren zur Aufzeichnung der elektrischen Aktivität des menschlichen Gehirns. Die aufgezeichneten Signale bestehen aus Spannungsschwankungen im Mikrovoltbereich, welche aus der synchronen elektrischen Aktivität von Neuronen resultieren [4]. Die Messung erfolgt über Elektroden, die auf der Kopfhaut platziert werden und die Potenzialdifferenzen zwischen verschiedenen Punkten erfassen [1].

EEG-Signale haben die Eigenschaft, extrem geringe Amplitude im Mikrovoltbereich zu besitzen, weshalb sie bei der Erfassung systematisch verstärkt werden müssen [1, 4]. Das Frequenzspektrum erstreckt sich dabei meist von 0,1 Hz bis 100 Hz [1]. In seiner Rohform ist ein EEG-Signal stark von Rauschen geprägt, weshalb es ein unzureichendes Signal-Rausch-Verhältnis (SNR) aufweist [4]. Da die erwarteten neuronalen Aktivitäten schwach sind, werden sie schnell von amplitudenstarken physiologischen Artefakten (wie z. B. Muskel- oder Augenbewegungen) sowie technischen Störeinflüssen überlagert, was zu Verzerrungen des Originalsignals führt [1].

2.2 Frequenzbänder

Frequenzbänder sind Frequenzbereiche, in welche die Gehirnwellen nach ihrer Frequenz eingeteilt werden. Die elektrischen Schwingungen werden in Hertz (Hz) gemessen, also in Ausschlägen pro Sekunde [1]. Die gezielte Auswertung spezifischer Frequenzbänder dient somit der fundierten Beurteilung von Gehirnaktivitäten [1].

Das EEG-Spektrum wird in jeweils 5 Frequenzbänder unterteilt [1]:

Tabelle 2.1: Die fünf EEG-Frequenzbänder mit ihren Frequenzbereichen und Zuständen. Diese Einteilung ist die Grundlage für die spätere Zuordnung der Leistungswerte [1]

Band	Frequenzbereich	Assoziierter Zustand
Delta	0,5–4 Hz	Tiefer, sehr entspannter und erholsamer Zustand
Theta	4–8 Hz	Leichte Schlafphase oder Meditation; Zustand zwischen Schlaf und Wachsein
Alpha	8–12 Hz	Ruhiger, stressfreier Zustand; in der Großhirnrinde deutlich nachweisbar, am ausgeprägtesten bei geschlossenen Augen
Beta	12–30 Hz	Aktives Denken und Problemlösen
Gamma	ab 30 Hz	Konzentration und Multitasking

2.3 Frequenzanalyse

Wichtig ist es zu bestimmen, welche Frequenzbänder in EEG-Signalen wie stark vertreten sind. Im Folgenden wird beschrieben, welche Verfahren dafür nötig sind.

2.3.1 Fourier-Transformation

Die Fast Fourier Transform (FFT) ist ein mathematisches Verfahren, das komplexe Signale in ihre sinusförmigen Bestandteile zerlegt und sie vom Zeitbereich in den Frequenzbereich überführt (Abbildung 2.1) [1]. Dies wird verwendet, um bei der Analyse der Gehirnströme die spektralen Eigenschaften von Signalen zu untersuchen und Frequenzspitzen (Peaks) zu berechnen [1].

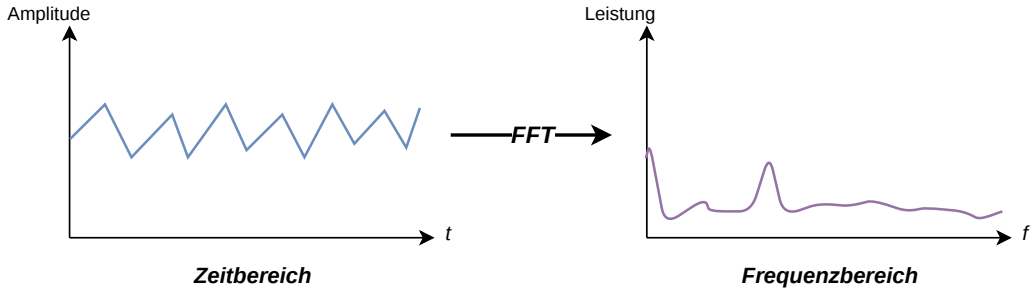


Abbildung 2.1: Überführung eines Signals vom Zeit- in den Frequenzbereich mittels FFT. Der Signalverlauf wird dabei in seine Frequenzanteile zerlegt (erstellt mit draw.io).

Das Ergebnis der FFT ist je Frequenz-Bin eine komplexe Zahl, bestehend aus einem Real- und einem Imaginärteil. Der Realteil beschreibt die Amplitude des Kosinus-Anteils und der Imaginärteil die Amplitude des Sinus-Anteils der jeweiligen Frequenz. Indem man, wie in der folgenden Gleichung, die Summe der quadrierten Real- und Imaginärteile berechnet, erhält man die Leistung eines Bins [35].

$$P = \mathrm{Re}^2 + \mathrm{Im}^2 \tag{2.1}$$

Ein zentrales Merkmal der FFT ist die Frequenzauflösung Δf, die angibt, wie genau die einzelnen Frequenzstützstellen unterteilt werden können. Die Auflösung ergibt sich aus der Abtastrate f_s und der Anzahl der Werte N pro Fenster [24]:

$$\Delta f = \frac{f_s}{N}. \tag{2.2}$$

Je größer das Fenster N gewählt wird, desto kleiner wird Δf und desto feiner ist folglich die Auflösung [23].

Daraus lässt sich ableiten, dass jeder Bin für eine bestimmte Frequenz steht.

2.3.2 Fensterung und spektrale Leckage

Die Fensterung ist ein Verfahren, um Signale zu analysieren, deren Frequenzen sich im Laufe der Zeit verändern [1]. Die Auswertung geschieht in zeitlich verschiebbaren Fenstern statt im gesamten Signal auf einmal [1]. Wenn ein Signal für die Datenverarbeitung abgeschnitten wird, entsteht eine spektrale Leckage. Dies geschieht häufig dann, wenn die Länge des erfassten Signalabschnitts kein exaktes Vielfaches der eigentlichen Signalperiode ist. Durch dieses harte Abschneiden wird die Energie des Signals im Frequenzspektrum verfälscht und auf einen breiteren Frequenzbereich verteilt, was zu Störfrequenzen führt. Um diese Leckage auszugleichen, nutzt man die Fensterfunktion, bei der man die Stärke des Signals an den Rändern des Fensters sanft abschwächt [23].

2.3.3 Welch-Methode

Um Frequenzspektren robuster zu schätzen und Rauschen zu reduzieren, wird die Welch-Methode verwendet [23]. Es ist ein Rechenverfahren, das in drei Schritten vorgeht:

1. Das ursprüngliche Signal wird in mehrere kürzere Fenster unterteilt, die sich teilweise überlappen dürfen [23].
2. Auf jedes Fenster wird eine Fensterfunktion angewendet, um dann für jedes Fenster einzeln das Spektrum zu berechnen [23].
3. Ganz am Ende werden die Einzelspektren zu einem Durchschnittswert gemittelt [23].

2.4 Lokale Sprachmodelle auf mobilen Geräten

Im folgenden Abschnitt werden die relevanten Grundlagen dargestellt, die für das Ausführen von Sprachmodellen auf mobilen Geräten notwendig sind. Zunächst werden große Sprachmodelle, deren Vor- und Nachteile sowie die Quantisierung vorgestellt.

2.4.1 Large Language Models

LLMs haben die Verarbeitung natürlicher Sprache (Natural Language Processing) maßgeblich verändert. LLMs sind komplex und basieren auf der Transformer-Architektur [38]. Das Besondere daran ist der Self-Attention-Mechanismus (Selbstaufmerksamkeit). Dieser dient dazu, globale Abhängigkeiten zwischen Ein- und Ausgabesequenzen effizient zu modellieren [36]. Die hohe Leistungsfähigkeit der Modelle wird durch die gelernten Werte von Milliarden von Parametern erreicht. Sie zeigen Stärken im Sprachverstehen, in der

Textsynthese, im logischen Schlussfolgern sowie in der generativen Texterstellung [38]. Dadurch entstehen Herausforderungen, die es zu bewältigen gilt, etwa wenn man sie auf ressourcenbeschränkten Endgeräten wie Smartphones (sogenannten Edge-Geräten) zum Laufen bringen möchte. So benötigen LLMs viel Speicherplatz und Rechenleistung [38].

In der maschinellen Sprachverarbeitung werden Texte in Token zerlegt, die als kleinste Informationseinheiten dienen. So wird im Fachartikel [21] gesagt: „Ein Token kann für ein einzelnes Zeichen, einen Teil eines Wortes (z. B. ein Suffix oder Präfix), ein ganzes Wort oder sogar eine kurze, aus mehreren Wörtern bestehende Phrase stehen." Damit die benötigte Rechenleistung reduziert wird, enthält jedes Token eine eindeutige numerische Identifikationsnummer (ID), anhand derer das Modell die Sprache lernt und verarbeitet [21].

Das Kontextfenster ist die maximale Anzahl an Token, die das Modell gleichzeitig verarbeiten kann [21]. Es ist sozusagen das Arbeitsgedächtnis des Modells. Sollte das Kontextfenster überschritten werden, werden die älteren Informationen gekürzt oder zusammengefasst, damit das Modell weiterhin im selben Kontext arbeiten kann [21]. Die Kontextgröße ist die maximale Summe der Eingabe- und Ausgabe-Token, die ein Modell verarbeiten kann [15].

2.4.2 On-Device-Inferenz und Quantisierung

On-Device-Inferenz beschreibt die lokale Ausführung von KI-Modellen direkt auf den lokalen Endgeräten (beispielsweise einem Smartphone), anstatt die Daten zur Verarbeitung an einen externen Cloud-Server zu senden [38].

Dieser Ansatz bietet daher drei wesentliche Vorteile:

- **Datenschutz:** Da die Daten das Gerät nicht verlassen, wird die Privatsphäre der Nutzer besser geschützt [38].
- **Offline-Fähigkeit:** Auch ohne Internetverbindung ist die Anwendung nutzbar [38].
- **Kostenersparnis:** Es fallen keine Kosten durch die Abhängigkeit von Cloud-Diensten an [38].

Neben den Vorteilen gibt es auch gewisse Herausforderungen, die man zu bewältigen hat:

- **Speicher und Rechenleistung:** Da LLMs sehr große Sprachmodelle sind, benötigen sie aufgrund ihrer hohen Anzahl an Parametern ausreichend großen Arbeitsspeicher (RAM) und Rechenkapazität. Oft ist dies mehr, als herkömmliche Smartphones bieten, was im Extremfall zu einem Speicherüberlauf führt, im Englischen als memory overflow oder Out of Memory (OOM) bezeichnet [17, 38]. Ein OOM bezeichnet den Zustand, in dem kein Arbeitsspeicher mehr für neue Objekte verfügbar ist [17].
- **Vielfalt der Endgeräte:** Verschiedene Endgeräte besitzen unterschiedliche Prozessoren und Spezialchips. Eine Lösung zu programmieren, die auf allen Endgeräten funktioniert, stellt daher eine Herausforderung dar [38].
- **Katastrophales Vergessen:** Ein häufiges und bekanntes Phänomen ist, dass ein Modell unweigerlich zuvor erlerntes Wissen vergisst, wenn es neue Informationen lernt [38].

- **Drosselung:** Ein weiteres Problem ist das Thermal Throttling, also ein Schutzmechanismus mobiler Geräte, der automatisch bei Überhitzung die Rechenleistung drosselt und dadurch die Latenz der Modelle erhöhen kann [39].

Es spielen nicht alle Punkte eine gleich große Rolle. Datenschutz und Offline-Fähigkeit sind hier keine offenen Fragen, sondern der Grund für die lokale Ausführung. Die Herausforderung liegt bei den übrigen Punkten: Speicher und Rechenleistung entscheiden darüber, ob, welches und wie ein Modell auf dem Testgerät läuft (Abschnitt 4.4.1), die Drosselung kann die gemessenen Latenzen beeinflussen [39] und die Gerätevielfalt kann die Übertragbarkeit begrenzen. Das katastrophale Vergessen spielt hierzu keine Rolle, da alle Modelle unverändert übernommen werden.

Damit diese Modelle auf Mobilgeräten laufen können, ist die sogenannte Quantisierung ein essenzielles Verfahren. Die Dateigröße des Modells wird reduziert, indem die Genauigkeit der gespeicherten Zahlenwerte künstlich verringert wird. Wechselt man beispielsweise von einer 32-Bit- auf eine 8-Bit-Speicherung, sinkt der Arbeitsspeicherbedarf und es kommt zu schnelleren Berechnungen auf dem Endgerät [38]. Dieser Gewinn an Speicher- und Recheneffizienz kann jedoch zu Leistungseinbußen und Genauigkeitsverlusten führen [38]. Sprachmodelle reagieren aufgrund ihrer Architektur empfindlich auf eine verringerte numerische Präzision [38].

2.4.3 Modell und Laufzeitumgebung

Der Unterschied zwischen einem Modell und einer Runtime (Laufzeitumgebung) lässt sich grundlegend wie folgt beschreiben:

- **Modell:** Ein Modell ist die Basis, die durch das Training Fähigkeiten wie Textverständnis, logisches Schließen oder die Generierung von Inhalten erlernt hat [38].
- **Runtime:** Die Runtime ist hingegen die Software, die das Modell auf dem Endgerät ausführt [15, 38]. Sie stellt die notwendigen Ressourcen und Schnittstellen bereit, um das Modell zu betreiben [38].

2.4.4 Systemintegrierte und frei deploybare Modelle

Grundsätzlich gibt es zwei Kategorien lokaler Sprachmodelle, die unterschieden werden können:

- **Systemintegrierte Modelle:** zum einen gibt es systemintegrierte Modelle, die tief in das Betriebssystem integriert sind. Ein Beispiel hierfür ist das Modell Gemini Nano, welches über den Android-Systemdienst AICore ausgeführt und lokal verwaltet wird [14]. Der architektonische Vorteil dieses Ansatzes besteht darin, dass Entwickler keine eigene Runtime bereitstellen müssen, sondern über standardisierte Programmierschnittstelles (APIs) auf das Modell zugreifen können [14]. Das Betriebssystem übernimmt dabei die zentrale Verwaltung sowie Aktualisierung, was den Speicher- und Ressourcenbedarf der einzelnen Applikation erheblich reduziert.
- **Frei deploybare Modelle:** zum anderen gibt es frei deploybare Modelle, die gemeinsam mit einer passenden Runtime als Paket von Entwicklern direkt auf das Endgerät installiert und in der Applikation ausgeführt werden müssen [19]. Dadurch kann

eine maximale Flexibilität gewährleistet werden, da sie unabhängiger von spezifischen Systemvorgaben oder vorinstallierten Systemdiensten funktionieren. Für die technische Umsetzung werden sogenannte „Task Bundles“ (Dateiformate wie .task oder .litertlm) genutzt [15]. Diese Datenpakete bündeln das eigentliche Modell mit den notwendigen Tokenizer-Parametern (Werkzeuge zur Textzerlegung), um eine reibungslose Ende-zu-Ende-Verarbeitung zu gewährleisten [15]. Dieser Ansatz erhöht jedoch den Speicherbedarf der Anwendung.

2.5 Brain-Computer-Interface (BCI)

Ein BCI, zu Deutsch Gehirn-Computer-Schnittstelle, beschreibt eine Technologie, die eine Kommunikation zwischen Gehirn und Computer (externem Gerät) ermöglicht [4]. Häufig gemessen durch EEG, werden mentale Zustände und Absichten in Echtzeit erkannt. Dadurch sind Steuerungsbefehle möglich, die ganz ohne Muskelkraft auskommen [4].
Ein BCI-System erfasst die Gehirndaten und verarbeitet diese Signale. Häufig geschieht dies in einem geschlossenen System, in dem der Nutzer auf Basis der Klassifizierung eine unmittelbare Rückmeldung erhält [4].

Vor allem körperlich stark beeinträchtigte Menschen profitieren von BCI-Systemen. Zu den bekannten Beispielen zählen [1, 4]:

- **Kommunikationssysteme:** Spezielle Systeme ermöglichen gelähmten Personen die Kommunikation mit ihrer Umwelt.
- **Bewegung und Steuerung:** BCI-Systeme erlauben die gedankliche Steuerung von Prothesen, Rollstühlen sowie das Steuern eines Cursors auf dem Computerbildschirm.
- **Assistenzsysteme:** Auch bei der Fehlerüberwachung können anhand der Gehirnaktivität die Absichten einer Notbremsung erkannt und ausgeführt werden, noch bevor das Bremspedal physisch betätigt wird.

Diese Arbeit setzt auf einem solchen EEG-basierten BCI auf und ergänzt es um ein lokales Sprachmodell, das die Messwerte in verständlichen Text überführt. Sie setzt erst nach der Aufnahme an und wertet bereits gespeicherte Sessions aus. Im folgenden Kapitel werden Modelle und Software-Frameworks vorgestellt, die für den mobilen Einsatz optimiert sind.

3 Stand der Technik

Während Kapitel 2 die allgemeinen Konzepte behandelt, werden in diesem Kapitel aktuell verfügbare Systeme betrachtet und ausgewählt, welche in dieser Arbeit verwendet werden. Zunächst werden die Lösungen für die Ausführung auf dem Endgerät gegenübergestellt, anschließend die verwendeten Sprachmodelle aus der Gemma-Familie sowie Qwen vorgestellt. Schließlich wird die bestehende Anwendung genauer erläutert und der Beitrag dieser Arbeit aufgezeigt.

3.1 On-Device-Inferenz-Lösungen

Um ein Sprachmodell lokal auf einem Mobilgerät auszuführen, wird eine Laufzeitumgebung benötigt. Dafür stehen unterschiedliche Lösungen zur Verfügung. Diese lassen sich, wie in Abschnitt 2.4.4 dargestellt, in systemintegrierte und frei deploybare Ansätze einteilen. Beide werden im Folgenden betrachtet.

3.1.1 Systemintegrierte Lösung

Das systemintegrierte Modell Gemini Nano ist stark an die Hardware gebunden, da AICore spezifische On-Device-Hardwarekomponenten nutzt, um die Inferenz zu beschleunigen [14]. Aus diesem Grund setzt Gemini Nano bestimmte von Google freigegebene Hardware voraus. Das für diese Arbeit verwendete Testgerät Samsung Galaxy S23 erfüllt diese Anforderungen nicht, sodass Gemini Nano nicht genutzt werden kann [18].

Die Möglichkeit einer virtuellen Testumgebung wie eines Android-Emulators ist ausgeschlossen, da die physische Präsenz der Gerätehardware benötigt wird [14, 38].

Aufgrund dieser Einschränkungen steht die systemintegrierte Lösung Gemini Nano für diese Arbeit nicht zur Verfügung. Deshalb wird im Folgenden die frei deploybare Lösung betrachtet, die auf dem Testgerät Samsung Galaxy S23 genutzt werden kann.

3.1.2 Frei deploybare Lösung

Die MediaPipe LLM Inference API ist eine von Google bereitgestellte, frei deploybare Lösung für die lokale Ausführung von Sprachmodellen auf mobilen Geräten [15].

Abhängig vom Modelltyp und den Einstellungen können die Berechnungen entweder auf dem Hauptprozessor (CPU) oder dem Grafikprozessor (GPU) des mobilen Geräts durchgeführt werden [15].

Die Schnittstelle unterstützt von Haus aus kompakte, speziell für mobile Geräte entwickelte Modelle. Dazu gehört insbesondere die Gemma-Familie von Google, etwa Gemma 3 1B und Gemma 2 2B [15]. Darüber hinaus lassen sich über die LiteRT-Community vorkonvertierte Modelle im MediaPipe-Format einbinden [15], darunter Qwen 2.5 1.5B [27]. Diese drei Modelle werden für den weiteren Verlauf der Arbeit verwendet und sind in Abbildung 3.1 zusammengefasst.

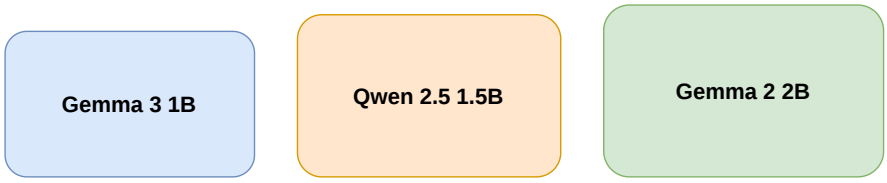


Abbildung 3.1: Übersicht der drei verwendeten Sprachmodelle. Sie unterscheiden sich in Größe und Hersteller, laufen aber über dieselbe Laufzeitumgebung (erstellt mit draw.io).

Unabhängig von der hier genutzten MediaPipe-Laufzeitumgebung gibt es weitere frei deploybare Modelle anderer Anbieter wie etwa DeepSeek-R1 [6] oder die Llama-Modelle von Meta [28].

Zusätzlich zu MediaPipe existieren weitere Laufzeitumgebungen, die auf Android-Geräten laufen. Beispiele sind llama.cpp oder MNN [19]. Nichtsdestotrotz wird in dieser Arbeit MediaPipe verwendet, da in diesem Fall mehrere Aspekte dafür sprechen. Die Bibliothek lässt sich zum einen als offizielle Android-Schnittstelle von Google direkt in die bestehende Java-Anwendung einbinden [15]. llama.cpp ist dagegen eine C++-Bibliothek [19], deren Einbindung in eine Java-Anwendung mit zusätzlichem Aufwand verbunden ist. Zudem liegen die Modelle im passenden `.task`-Format vor [15, 27], sodass sich alle drei Modelle unter derselben Laufzeitumgebung mit identischen Parametern ausführen lassen. Die Kombination aus der einfachen Integration und den identischen Parametern der verfügbaren Modelle hat schlussendlich diese Entscheidung treffen lassen.

3.2 Die Gemma-Modellfamilie

Die Gemma-Modellfamilie gehört zu den frei deploybaren und ressourcenschonenden Modellen, die vom Gemma-Team bei Google DeepMind entwickelt wurden [11].

Die Gemma-Modelle besitzen mehrere besondere Merkmale. Neben der freien Zugänglichkeit der Modelle zeichnen sie sich durch eine ressourcenschonende Ausführung auf herkömmlicher Standardhardware wie Smartphones aus [10, 11]. Um unterschiedlichen technischen Anforderungen gerecht zu werden, sind die Modelle zudem in mehreren Größen verfügbar. Diese werden in Milliarden von Parametern gemessen. So bietet die zweite Generation (Gemma 2) Varianten mit 2, 9 und 27 Milliarden Parametern [10]. Die dritte Generation (Gemma 3) erweitert diese Skalierung auf Modelle mit 1, 4, 12 und 27 Milliarden Parametern [11]. Die aktuellste Generation (Gemma 4) bietet sogar Modelle mit bis zu 31 Milliarden Parametern [12].

Auch der Funktionsumfang wurde erweitert. Modelle vor der dritten Generation (Gemma 1–2) unterstützten nur reine Textverarbeitung. Seit der dritten Generation sind auch

multimodale Eingaben möglich, sodass neben Text auch Bilder verarbeitet werden können [11]. Zudem wurden die Fähigkeiten zur Verarbeitung unterschiedlicher Sprachen sowie zum Erfassen deutlich längerer Textzusammenhänge verbessert [11]. Die neueste Version Gemma 4 unterstützt noch größere Kontextfenster von bis zu 256.000 Tokens und besitzt einen integrierten „Thinking-Modus“ für logisches Schlussfolgern [12]. Neben der allgemeinen Basisversion gibt es die Modelle auch in einer speziell angepassten Form, die gezielt für das Führen von Dialogen und das präzise Befolgen von Anweisungen trainiert wurde [10, 11]. Gemma 2 und Gemma 3 verwenden einen SentencePiece-Tokenizer mit jeweils ca. 256.000 und 262.000 Vokabulareinträgen [10, 11].

3.3 Die Qwen-Modellfamilie

Als drittes Modell wird Qwen des Herstellers Alibaba herangezogen. Genauso wie die Gemma-Modelle ist es frei deploybar. Jedoch stammt es von einem anderen Hersteller und gehört damit nicht zur selben Modellfamilie [31]. Die Generation Qwen2.5 ist in fein abgestuften Konfigurationen von 0,5 bis 72 Milliarden Parametern verfügbar, wobei die kompakten Varianten gezielt für ressourcenbeschränkte Edge-Szenarien und mobile Endgeräte konzipiert sind [31]. Mit 1,5 Milliarden Parametern liegt die in dieser Arbeit verwendete Variante von Qwen genau zwischen Gemma 3 1B und Gemma 2 2B. Qwen 2.5 nutzt hingegen Byte-Level Byte-Pair Encoding (BBPE) mit einem Vokabular von 151.643 regulären Tokens [31] und damit ein anderes Verfahren sowie ein deutlich kleineres Vokabular als die Modelle von Google. Die neueste Generation der Familie ist Qwen3, die einen Thinking-Modus und einen Non-Thinking-Modus in einem einheitlichen Modell vereint [32].

3.4 Bestehende Anwendung

Im Folgenden wird die bestehende Anwendung vorgestellt, die als Grundlage für die Implementierung der neuen Funktionalität dient.

3.4.1 Unicorn Hybrid Black

Das Unicorn Hybrid Black ist die BCI-Plattform, die in dieser Arbeit verwendet wird. Es ist eine mobile Brain-Computer-Interface-Plattform der Firma g.tec [9]. Das Aufnahmegerät erfasst die elektrische Aktivität des Gehirns über acht Elektroden, die auf der Kopfhaut angebracht werden. Die Abtastrate beträgt genau 250 Hz [8]. Besonders ist die Kompaktheit des Geräts. Durch eine tragbare, drahtlose Bauform eignet es sich für den mobilen Einsatz. Die in Abbildung 3.2 markierten Komponenten des Geräts wurden für die Messungen dieser Arbeit verwendet.

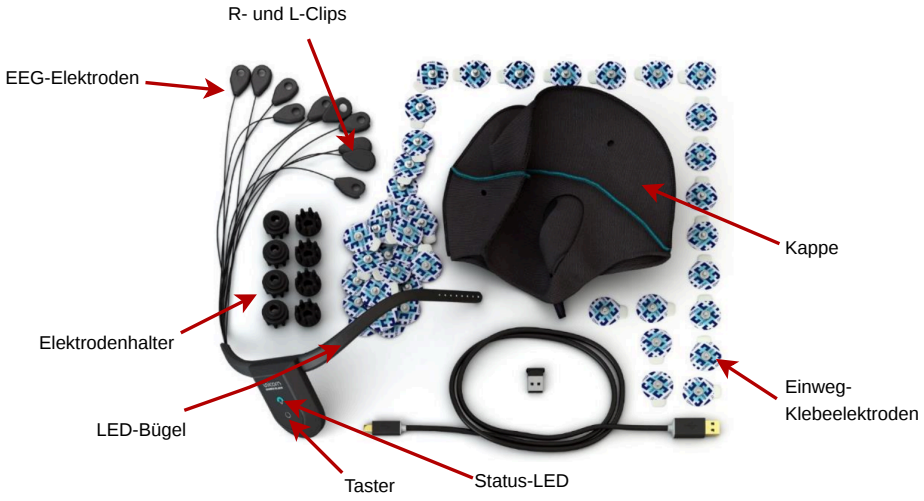


Abbildung 3.2: Komponenten des verwendeten EEG-Geräts g.tec Unicorn Hybrid Black (eigene Beschriftung, erstellt mit draw.io) [9].

3.4.2 Datenübertragung und Anbindung

Die Übertragung der Messdaten an das Endgerät erfolgt drahtlos über Bluetooth, wobei jedes Sample aus den jeweils folgenden 17 Werten besteht [7].

Tabelle 3.1: Aufbau eines Samples des Unicorn Hybrid Black. Von den 17 Werten werden nur die acht EEG-Kanäle und das Gültigkeitskennzeichen ausgewertet [7]

Werte	Bezeichnung	Anzahl
1–8	EEG-Kanäle	8
9–11	Beschleunigungssensor (x, y, z)	3
12–14	Gyroskop (x, y, z)	3
15	Batteriestand	1
16	Counter (fortlaufender Zähler)	1
17	Validation Indicator (valid-Flag)	1
	Gesamt	**17**

3.4.3 Funktionsumfang der Anwendung

Die als Grundlage für diese Arbeit genutzte Anwendung ist das Ergebnis eines Projektes aus einem vorangegangenen Praktikum. Sie bildet deren Basis, indem sie eine Verbindung mit dem Unicorn Hybrid Black aufbaut, EEG-Daten aufnimmt und speichert. Diese Daten werden zugleich als Zeitverlauf oder als Live-FFT visualisiert. Die Daten werden dauerhaft gespeichert und können zusätzlich als .db- oder .json-Datei exportiert werden. Der Datenpfad verläuft dabei vom Gerät über Bluetooth zu einem Hintergrunddienst, der die eingehenden Samples in der Datenbank ablegt.

3.4.4 Beitrag dieser Arbeit

Sprachmodelle werden immer häufiger für die Auswertung von EEG-Daten eingesetzt, unter anderem zur Erzeugung von Befundtexten oder zur Deutung von Signalen [3]. Als offene Herausforderung wird dabei die Ausführung auf Endgeräten mit geringer Latenz genannt [3]. Zusätzlich müssen für einen praktischen Einsatz der Speicherbedarf, die Stabilität und die Qualität der Ausgabe, also des erzeugten Textes, angemessen ausfallen.

Es gibt hierzu unterschiedliche Ansätze, wie vorgegangen wird. Chen u. a. betreiben ein Sprachmodell lokal, passen es dafür aber durch Pruning und Fine-Tuning an [5]. Murhekar u. a. belassen nur ein kleines Vorverarbeitungsmodul auf dem Gerät und lagern das Sprachmodell aus [29]. EEG-Anwendungen, die speziell auf Android ausgeführt werden, laufen jedoch ohne Sprachmodell [4, 20]. Umgekehrt untersuchen Messstudien zu Sprachmodellen auf Mobilgeräten allgemeine Aufgaben und keine EEG-Daten [19].

Keiner dieser Ansätze verbindet die lokale Ausführung eines unveränderten Sprachmodells mit aufbereiteten Merkmalen als Eingabe (Tabelle 3.2). Als Anpassung gilt dabei eine Veränderung der Modellgewichte. Die Wahl einer bereits quantisiert veröffentlichten Variante sowie Laufzeitparameter wie Kontextgröße oder Top-K zählen nicht dazu, da sie die Gewichte unberührt lassen.

Tabelle 3.2: Gegenüberstellung nach Ausführungsort des Sprachmodells, Anpassung des Modells, Art der Eingabe sowie Vergleich mehrerer Modelle auf dem Endgerät. Als Anpassung gilt eine Veränderung der Gewichte des Sprachmodells durch Fine-Tuning oder Pruning. Ein Strich bedeutet, dass kein Sprachmodell verwendet wird.

Arbeit	**Modell lokal**	**Sprachmodell angepasst**	**Eingabe**	**Vergleich**
Chen u. a. [5]	ja	ja	Rohsignale	nein
Murhekar u. a. [29]	nein	nein	Rohsignale	nein
Blum u. a. [4]	kein Sprachmodell	—	Rohsignale	nein
Hazarika u. a. [20]	kein Sprachmodell	—	Rohsignale	nein
Guo u. a. [19]	ja	nein	kein EEG	ja
Diese Arbeit	ja	nein	Merkmale	ja

Die Frage bleibt damit, ob unveränderte, frei verfügbare Sprachmodelle ausreichen, um aufbereitete EEG-Merkmale analysieren zu können. Der Beitrag dieser Arbeit ist daher der systematische Vergleich dreier solcher Modelle unter identischen Bedingungen anhand von Latenz, Ressourcenverbrauch, Stabilität und Ausgabequalität.

Die bestehende Anwendung wird um neue Funktionalitäten erweitert. Diese bestehen aus der Verarbeitung der EEG-Daten durch lokale Sprachmodelle. Umgesetzt wird dies in drei neuen Kernkomponenten:

- Schicht zur Merkmalsextraktion (`features`).
- Anbindung eines oder mehrerer Sprachmodelle (`llm`).
- Benutzeroberfläche zur KI-gestützten Analyse (`analyse`).

Darüber hinaus wird für die in der Evaluation benötigten Messungen ein Benchmark implementiert, bei dem systematisch und automatisiert gemessen wird (`benchmark`). Im Originalcode wurden nur minimale Änderungen vorgenommen.

4 Methodik

In diesem Kapitel geht es um die Methodik der Arbeit. Dabei wird als Erstes die Merkmalsextraktion näher erläutert, bei der die rohen EEG-Daten in kompakte Kennzahlen überführt werden. Anschließend wird die Modellauswahl beschrieben und die Anwendungsszenarien definiert. Abschließend folgt das Messprotokoll, das die Vorgehensweise der Messungen und die erfassten Metriken beschreibt.

4.1 Merkmalsextraktion

Die Merkmalsextraktion ist ein zentraler Bestandteil der Arbeit. Ein Sprachmodell kann keine EEG-Rohdaten verarbeiten, da bei einer Abtastrate von 250 Hz und acht Kanälen bereits in kurzer Zeit eine große, unstrukturierte Zahlenmenge entsteht. Bei einer nur einminütigen Aufnahme ergeben sich beispielsweise

$$\underbrace{8}_{\text{Kanäle}} \times \underbrace{250}_{\text{Hz}} \times \underbrace{60}_{\text{Sekunden}} = 120\,000 \text{ Werte.} \tag{4.1}$$

Diese Datenmenge muss zunächst in eine kompakte Form umgewandelt werden (Abbildung 4.1), weil sie ansonsten den Kontext des Modells überschreitet und keine fachliche Bedeutung transportiert. Die daraus gewonnenen aggregierten Merkmale sind interpretierbar, token-sparsamer und können vom Sprachmodell verarbeitet werden.



Abbildung 4.1: Datenfluss der Merkmalsextraktion von den Rohdaten bis zum Text. Jede Stufe verdichtet die Daten weiter, bis eine für das Modell nutzbare Beschreibung entsteht (erstellt mit draw.io).

4.1.1 Segmentierung

Würden die Merkmale als ein einziger Mittelwert über die gesamte Aufnahme berechnet, ginge der zeitliche Verlauf der Session verloren. Aus diesem Grund wird jede Session in mehrere gleich große Segmente aufgeteilt (Abbildung 4.2). Da das Signal mit einer konstanten Abtastrate von 250 Hz aufgenommen wird, entspricht eine gleiche Anzahl an Samples automatisch einer gleichen Zeitdauer. Die Segmentierung wird daher Sample-basiert durchgeführt. Dadurch ist es leichter nachvollziehbar, wie sich die Signalcharakteristik zwischen Beginn und Ende der Aufnahme verändert. Durch dieses Verfahren wird die Analyse robuster gegenüber Artefakten, da diese nur einen Teil der Session betreffen und nicht die gesamte Aufnahme verfälschen. Die Wahl auf genau vier Segmente ist hierbei ein Kompromiss. Sie entstand durch eine vertretbare Überlegung, niedrige Komplexität mit einem feinen zeitlichen Verlauf zu kombinieren. Mehr Segmente würden den aufbereiteten Text für die Modelle vergrößern, da je Segment ein eigener Block entsteht (Abschnitt 4.1.4), und somit die Anzahl der Eingabe-Tokens erhöhen. Weniger Segmente könnten zu einem zu groben Zeitverlauf führen.

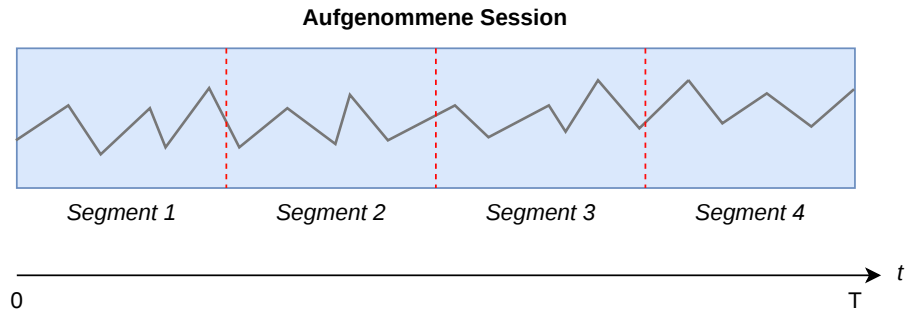


Abbildung 4.2: Aufteilung einer Session in vier gleich lange Segmente. So bleibt der zeitliche Verlauf erkennbar, statt nur einen Mittelwert zu bilden (erstellt mit draw.io).

Damit die Aufteilung der Session möglich ist, muss die Aufnahme eine Mindestlänge haben. Damit pro Segment mindestens ein vollständiges Analysefenster für die spektrale Berechnung zur Verfügung steht, sind bei vier Segmenten und einer Fenstergröße von 256 Werten mindestens

$$4 \text{ Segmente} \times 256 \text{ Werte (Fenster)} = 1024 \text{ Samples} \approx 4 \text{ s} \tag{4.2}$$

erforderlich. Wenn die Aufnahme zu kurz ist, um die Mindestlänge zu erfüllen, wird die Analyse abgebrochen und eine entsprechende Meldung angezeigt.

Die Fenstergröße ist kein fester, unveränderlicher Wert, sondern variabel einstellbar. Eine größere Fenstergröße erhöht die Frequenzauflösung [23], erfordert jedoch eine längere Mindestdauer der Session. Für alle folgenden Untersuchungen wird mit der Standardfenstergröße von 256 Samples gearbeitet. Dies liegt an der Fourier-Transformation, bei der Zweierpotenzen für den Wert verwendet werden. Dieser Wert liegt nahe der Abtastrate von 250 Hz des Unicorn Hybrid Black und entspricht damit einer Dauer von etwa einer Sekunde. Nach Gleichung 2.2 ergibt sich daraus eine Frequenzauflösung von rund 1 Hz, womit sich die fünf Frequenzbänder aus Tabelle 2.1 noch trennen lassen. Dagegen würde eine Fenstergröße von 128 Samples eine Frequenzauflösung von etwa 2 Hz ergeben und damit halbieren, was einen Verlust an Genauigkeit zur Folge hat. Bei 512 Samples würde sich die Mindestdauer verdoppeln, was jedoch zu weniger Fenstern je Segment führt.

4.1.2 Spektrale Merkmale

Die spektralen Merkmale werden für jedes Segment bestimmt, indem die Signalenergie auf die einzelnen Frequenzbänder verteilt wird. Die kanalweise Berechnung erfolgt dabei mit der Welch-Methode [23]. Hierfür wird das Signal eines Segments in überlappende Fenster unterteilt (Abbildung 4.3); im vorliegenden Fall beträgt die Fenstergröße (Standard: 256) Samples. Für jedes dieser Fenster wird das Leistungsspektrum berechnet, das anschließend über alle Fenster gemittelt wird. Durch diese Mittelung wird das resultierende Spektrum deutlich stabiler und weniger rauschanfällig als bei einer einzelnen FFT. Allerdings ist dann mit einer geringeren Frequenzauflösung zu rechnen [23].

Um den Gleichanteil (0 Hz) des Signals zu eliminieren, wird vor der Fourier-Transformation zunächst der Mittelwert von jedem Segment subtrahiert. Anschließend wird die Hann-Fensterfunktion (auch Hanning genannt) verwendet, um jedes Fenster zu gewichten [23]. Die Gewichtung wird verwendet, da es ansonsten zu einem direkten, abrupten Abschneiden des Signals kommt, was zu harten Übergängen an den Fenstergrenzen führt [23]. Durch diesen Vorgang entsteht eine spektrale Leckage. Die Hann-Funktion hilft gegen diese Leckage, indem sie die Amplitudenwerte an den Rändern des Fensters sanft abschwächt [23].

Die Fensterfunktion wird mathematisch wie folgt beschrieben:

$$w(n) = 0.5 - 0.5 \cos\left(\frac{2\pi n}{N-1}\right) \tag{4.3}$$

wobei N die Fenstergröße (Standard: 256) bezeichnet [23].

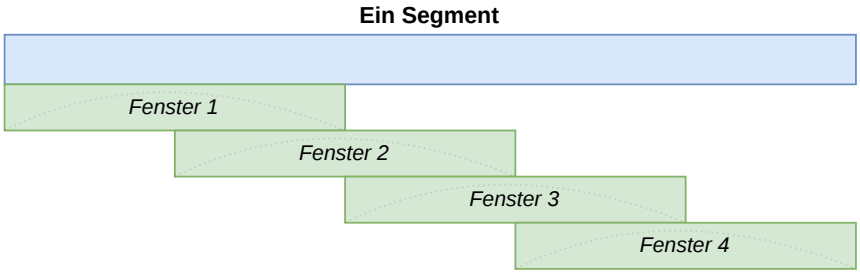


Abbildung 4.3: Zerlegung eines Segments in überlappende, mit einer Hann-Funktion gewichtete Analysefenster (50 % Überlappung) für die Welch-Methode (erstellt mit draw.io).

Jedes Fenster wird per FFT in den Frequenzbereich überführt, um das jeweilige lokale Leistungsspektrum (Periodogramm) zu berechnen [1, 23]. Die Leistung der einzelnen Frequenz-Bins wird den fünf Frequenzbändern Delta, Theta, Alpha, Beta und Gamma zugeordnet [1]. Dies erfolgt durch das Aufsummieren der Leistung aller Frequenz-Bins innerhalb der jeweiligen Bandgrenzen. In der Regel schwanken die absoluten Leistungswerte stark und sind schwer interpretierbar. Daher wird die relative Bandleistung verwendet, die sich aus dem Verhältnis der Leistung eines Frequenzbandes zur Gesamtleistung ergibt. Dadurch lässt sich unmittelbar und robust ablesen, welches Frequenzband im untersuchten Segment dominiert [1].

4.1.3 Zeitstatistische Merkmale

Neben den spektralen Merkmalen gibt es auch die zeitstatistischen Merkmale. Sie werden für jedes Segment berechnet [1]. Hierfür ist keine FFT notwendig, da die Merkmale

direkt aus dem Zeitsignal extrahiert werden [1]. Ein zentrales Merkmal ist die Spannweite (Peak-to-Peak-Amplitude), welche die Differenz zwischen dem größten und dem kleinsten Amplitudenwert beschreibt [22]. Der Vorteil dieser Herangehensweise ist, dass sie im Gegensatz zur Betrachtung eines absoluten Maximalwertes die Unabhängigkeit von einem potenziellen Gleichanteil des Signals gewährleistet. Aus diesem Grund stellt die Peak-to-Peak-Amplitude ein robustes Kriterium dar, um extreme Signalausschläge und Drifts zu identifizieren, wie sie bei Artefakten auftreten können [22].

Des Weiteren wird zur Beurteilung der Übertragungsqualität der Anteil der ungültigen Messwerte betrachtet. Dies ist der Anteil der Datenpunkte, die nicht empfangen wurden und deshalb bei der Übertragung durch eine Kopie des letzten Samples ersetzt und als ungültig markiert wurden. Man kann durch diesen Indikator erkennen, wie lückenhaft eine Aufnahme ist, wobei er keine Aussage über die Qualität des abgeleiteten Signals trifft.

4.1.4 Aufbereitung für das Sprachmodell

Im letzten Schritt der Merkmalsextraktion werden die berechneten Merkmale in eine textuelle Form gebracht, die vom Sprachmodell verarbeitet werden kann. Dabei enthält der Text folgende Informationen:

- aggregierte Kennzahlen
- relative Bandleistung der fünf Frequenzbänder (pro Segment)
- Spannweite (Peak-to-Peak-Amplitude)
- Anteil der ungültigen Messwerte

Die Merkmale werden neben der zeitlichen Aufteilung über die acht Kanäle zusammengefasst. Dabei wird die Bandleistung je Segment über alle acht Kanäle summiert und dann relativ zur Gesamtleistung gesetzt. Bei der Spannweite selbst wird nicht vorher gemittelt. Stattdessen wird der Kanal mit dem größten Ausschlag genannt, da sonst einzelne starke Artefakte verdeckt werden könnten.

Dabei werden bewusst keine Bewertungen oder Interpretationen in den Text aufgenommen. Alle Informationen sind objektive Messwerte, deren Bedeutung allein dem Sprachmodell überlassen wird. Dadurch wird gewährleistet, dass die eigentliche Analysearbeit vom Modell erledigt wird.

Der aufbereitete Text der gesamten Session mit je vier Blöcken wird dem Modell mit einem einzigen Aufruf übergeben. Die Option, jeden Block einzeln zu übergeben, wurde bewusst vermieden. Die Idee ist es, dass das Modell die gesamte Session gemeinsam und auf einen Schlag interpretiert. Es sollen keine schrittweisen Aussagen getroffen werden. Zudem müsste man alle Teilantworten zusammenschließen, was einen vermehrten Aufwand darstellt. Außerdem beziehen sich die Latenzmetriken so auf genau einen Aufruf und bleiben eindeutig.

4.2 Modellauswahl

Wie schon in Abschnitt 3.1.1 beschrieben, stehen systemintegrierte Modelle auf dem verwendeten Testgerät nicht zur Verfügung. Daher konzentriert sich diese Arbeit auf frei deploybare Modelle, wie sie in Abschnitt 3.2 und Abschnitt 3.3 beschrieben sind.

Um eine geeignete Evaluierung zu ermöglichen, wird der Einfluss der Modellgröße unter konstanten Ausführungsbedingungen untersucht. Dazu werden drei Modelle vergleichend betrachtet: zum einen die kompakte Variante Gemma 3 1B [11, 15], zum anderen die parameterstärkere Gemma 2 2B als größeres Referenzmodell [10, 15], sowie das Modell Qwen 2.5 1.5B des Herstellers Alibaba [15, 31], dessen Größe zwischen den beiden Gemma-Modellen liegt. Da alle drei Modelle nativ über die MediaPipe-Runtime (LLM Inference API) laufen [15], lässt sich bei gleicher Laufzeitumgebung sowohl der Effekt der Modellgröße als auch der Einfluss des Herstellers auf die Systemperformanz untersuchen.

Alle drei Modelle werden in 8-Bit-Quantisierung und mit identischer Kontextgröße (1280 Token) ausgeführt, sodass sich der Vergleich einzig auf die Modellgröße und den Hersteller bezieht. Die Wahl einer Kontextgröße von 1280 Token ergibt sich aus den verfügbaren Modellvarianten. Für Gemma 3 1B und Qwen 2.5 1.5B liegen zwar auch Varianten mit größeren Kontextfenstern vor, für Gemma 2 2B dagegen ausschließlich die Variante mit 1280 Token [25–27]. Damit ist 1280 der größte Wert, der für alle drei Modelle einheitlich zur Verfügung steht. Zudem wird die Token-Auswahl über einen festen Top-K-Wert von 64 begrenzt, was bedeutet, dass immer nur die 64 wahrscheinlichsten Token als Kandidaten betrachtet werden. Im Code wird nicht festgelegt, ob die Modelle auf der CPU oder GPU laufen. Daher führt MediaPipe alle Modelle standardmäßig auf dem Standard-Backend aus. Wie die Logcat-Ausgabe beim Laden zeigt, ist dies die CPU (über den XNNPACK-Delegate von TensorFlow Lite).

4.3 Anwendungsszenarien

Um die Eignung der Modelle für die Analyse von EEG-Daten zu untersuchen, werden drei Anwendungsszenarien betrachtet. Dabei wird untersucht, wie ein Sprachmodell die extrahierten Merkmale einer oder mehrerer Sessions verarbeitet. Jedes Szenario besteht aus einem einmaligen Aufruf des Modells, der die Merkmale der Session(s) als Eingabe erhält. Es wird keine vorab bekannte Musterlösung benötigt. Die Szenarien orientieren sich an zwei Aufgabenklassen, zum einen an der automatischen Vorstrukturierung von Daten, zum anderen an der Unterstützung einfacher Analyseprozesse.

- **Session-Report (Vorstrukturierung):** Aus den Merkmalen einer Session wird ein zusammenhängender, lesbarer Bericht generiert.
- **Artefakt-Erklärung (Analyseunterstützung):** Das Modell erhält die Kennzahlen einer Session und erklärt auffällige Muster (z. B. hohe Peak-to-Peak-Amplituden oder einen hohen Anteil ungültiger Werte) in natürlicher Sprache.
- **Session-Vergleich (Analyseunterstützung):** Es werden die Merkmale zweier Sessions übergeben und darauffolgend verglichen.

Der Prompt-Aufbau ist in allen drei Szenarien und Modellen identisch. Er beginnt mit der szenariospezifischen Anweisung, gefolgt von einem textuell aufbereiteten Merkmalssatz.

Das Niveau steigt dabei mit jedem Szenario. Während der Report lediglich ein Zusammenfassen erfordert, ist beim Artefakt-Erklärungsszenario zusätzlich das Deuten notwendig. Das Vergleichsszenario erfordert schließlich ein Schlussfolgern über zwei Datensätze hinweg. Dadurch kann man die Grenzen der Modelle in Bezug auf die Komplexität der Aufgabenstellung untersuchen.

Dabei wurde bewusst auf drei verschiedene Szenarien gesetzt, deren Fokus das Generieren von textuellen Inhalten ist. Sie unterstützen dabei, die EEG-Merkmale zu verstehen und in eine für Benutzer verständliche Sprache zu übersetzen. Auf Klassifikationsszenarien wurde bewusst verzichtet, da man für diese Vergleichswerte benötigt, die in dieser Arbeit nicht zur Verfügung stehen.

4.4 Messprotokoll (Benchmark)

Eine verlässliche Messung kann nur systematisch und unter kontrollierten Bedingungen ausgeführt werden. Da es sich um mehrere Messungen handelt, werden sie in kurzer Zeit automatisiert über alle Modell-Szenario-Kombinationen durchgeführt und in einer Comma-Separated Values (CSV)-Datei protokolliert. So wird sichergestellt, dass die Messungen eine reproduzierbare und auswertbare Datenbasis für die anschließende Evaluation liefern. Bevor die eigentliche Messung beginnt, wird einmalig eine Aufwärmphase durchgeführt, um Kaltstart-Effekte zu vermeiden. Ein solcher Kaltstart äußert sich darin, dass der erste Lauf eines Modells durch das Laden und Initialisieren deutlich länger ausfällt als die nachfolgenden. Dadurch wird verhindert, dass die erste Kombination verzerrt gemessen wird. Das Ergebnis dieser Messung wird nicht protokolliert und dient allein der Aufwärmung. Alle Messungen richten sich nach einer bestimmten Reihenfolge. Diese startet mit Gemma 3 1B, gefolgt von Gemma 2 2B, und endet mit Qwen 2.5 1.5B, jeweils in den Szenarien Report, Artefakt-Erklärung und Vergleich. Die Reihenfolge ist bewusst festgelegt und bleibt über alle Durchläufe gleich. So wird gewährleistet, dass jedes Modell unter identischen Bedingungen im Ablauf gemessen werden kann und die Messreihe reproduzierbar bleibt. Eine zufällige Reihenfolge würde zwar Reihenfolgeeffekte auf alle Modelle verteilen, erfordert jedoch transparentere Angaben, da sich die Effekte sonst nicht mehr nachvollziehen lassen.

Es wird jede Kombination von Modell und Anwendungsszenario N = 5 Mal gemessen, da Temperatur, Hintergrundlast oder Caching die Laufzeit von Lauf zu Lauf erheblich beeinträchtigen können [19]. Dabei wird die erste Messung jeder möglichen Kombination aus demselben Grund verworfen. Die zuvor beschriebene Aufwärm-Messung betrifft dagegen nur den einmaligen Systemstart. Die verbleibenden Werte werden gemittelt. Zusätzlich wird die Standardabweichung berechnet, die die Streuung der Messwerte um den Mittelwert beschreibt. Der Mittelwert $\bar{x}$ und die Standardabweichung s über die n einbezogenen Messwerte ergeben sich dabei wie folgt:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2} \tag{4.4}$$

Dadurch wird die Stabilität der Messungen sichtbar. Ein hoher Wert deutet auf eine starke Streuung hin, während ein niedriger Wert eine geringe Streuung und damit eine hohe Stabilität der Messungen anzeigt.

Die anfängliche Aufwärm-Messung wird von der Anwendung selbst ausgeführt. Das Verwerfen der ersten Messung je Kombination, die Mittelung der verbleibenden Messungen

sowie die Berechnung der Standardabweichung erfolgen dagegen nicht in der Anwendung. Sie werden stattdessen manuell auf Grundlage der in der CSV-Datei gespeicherten Rohwerte durchgeführt.

4.4.1 Metriken

Zur Bewertung der Modelle werden mehrere Metriken erfasst, die sich in Latenz, Ressourcenverbrauch und Stabilität gliedern. Dazu zählen unter anderem die Zeit bis zum ersten Token (TTFT) und der Speicherverbrauch als Proportional Set Size (PSS).

Tabelle 4.1: Übersicht der erfassten Metriken nach Latenz, Ressourcenverbrauch, Stabilität und Qualität. Bis auf die Ausgabequalität werden alle Werte automatisiert erhoben.

Kategorie	**Metrik**	**Beschreibung**
Latenz	Ladezeit	Zeit zum Laden des Modells
	TTFT	Zeit bis zum ersten erzeugten Token
	Gesamtdauer	Dauer der gesamten Inferenz
	Tokens/s	Durchsatz, unabhängig von der Ausgabelänge
Ressourcen	RAM (PSS)	Speicherverbrauch inkl. nativem Speicher
	Akku	Energieverbrauch (µAh) über eine Messreihe
Stabilität	Status	erfolgreich oder Fehler
Qualität	Ausgabequalität	qualitative Bewertung (Kapitel 6)

Die Tokens pro Sekunde erlauben einen fairen Vergleich unabhängig von der Länge der Ausgabe. Eine reine Zeitmessung würde längere Antworten offensichtlich benachteiligen. Die Metrik erhält man, wie in folgender Gleichung dargestellt, indem man die Anzahl der erzeugten Token durch die gesamte Inferenzdauer teilt:

$$\text{Tokens/s} = \frac{N_{\text{Token}}}{t_{\text{Inferenz}}} \tag{4.5}$$

wobei N_{Token} die Anzahl der erzeugten Token und t_{Inferenz} die Inferenzdauer in Sekunden bezeichnet.

Der Speicherverbrauch wird über die PSS erfasst, da diese exklusiv von der Applikation genutzte Speicherseiten sowie den proportionalen Anteil an gemeinsam verwendeten Seiten (Shared Pages) berücksichtigt. Dadurch eignet sie sich zur präzisen Bestimmung des tatsächlichen Speicherbedarfs, da durch sie gemeinsam genutzte Ressourcen (wie beispielsweise Bibliotheken oder Systemdienste) nicht mehrfach gezählt werden [16]. Der Wert wird direkt nach dem Abschluss einer Inferenz ermittelt, während das Modell selbst noch geladen ist. Es handelt sich um einen aktuellen Wert, bei dem keine Speicherbereinigung ausgelöst wird. Bereiche, die noch nicht freigegeben wurden, werden mit eingeschlossen. Dabei sollten die erfassten Werte als obere Schranke des tatsächlichen Bedarfs behandelt werden.

Der Akkuverbrauch wird über den BatteryManager des Geräts vor und nach einer Messreihe ermittelt. Die Messung erfolgt in Mikroamperestunden (µAh), da diese eine feinere Auflösung als die Angabe in Prozent bietet [13].

Im Gegensatz zum Benchmark, bei dem es gleich fünf Messungen gibt, werden für die reine Analyse einer Session nur Latenzmetriken erfasst. Der Grund liegt hierbei an der Menge der Messungen. Der Energieverbrauch lässt sich nur über eine ganze Messreihe

sinnvoll bestimmen, da er sich erst aus der Differenz des Akkustands über mehrere aufeinanderfolgende Durchläufe ergibt. Zudem schwankt der Arbeitsspeicherbedarf stark [16], sodass eine Messung keine verlässliche Aussage über den Speicherbedarf zulässt. Die Metriken Status und Ausgabequalität sind nur für die Evaluation relevant und werden daher nicht in der Analyse erfasst.

Bei der Ausgabequalität geht es um die Qualität der vom Modell gegebenen Antwort. Hierfür wird keine automatisierte Messung benötigt, da es keine feste Musterlösung gibt, mit der man sie vergleichen könnte. Ein eindeutiges Richtig oder Falsch existiert nicht. Stattdessen wird der Inhalt der Ausgabe anhand der folgenden vier Kriterien bewertet:

- **Korrektheit:** Stimmt die Aussage mit den extrahierten Merkmalen überein?
- **Halluzinationsfreiheit:** Werden keine Werte oder Begriffe erfunden, die nicht in den Daten stehen?
- **Verständlichkeit:** Ist die Ausgabe sprachlich klar und lesbar?
- **Formattreue:** Hält die Ausgabe die Vorgaben ein (Fließtext, kein Markdown, knappe Länge)?

4.4.2 Messbedingungen

Damit die Messungen unter vergleichbaren und fairen Bedingungen erfolgen können, werden sie unter den folgenden Rahmenbedingungen durchgeführt:

- Der Flugmodus im Testgerät ist aktiviert, um Netzwerkaktivitäten auszuschließen.
- Die Bildschirmhelligkeit wird auf einen festen Wert (ungefähr 50 %) eingestellt, um den Akkuverbrauch über alle Messungen hinweg vergleichbar zu halten.
- Das Handy befindet sich nicht im Ladezustand, da dies den Akkuverbrauch während der Messung verfälschen würde. Bei Start der Messung befindet sich der Akkustand bei ungefähr 60 %. Dieser Wert wird während aller Messungen nicht gehalten, sondern der Verbrauch wird ab der ersten Messung festgehalten. Dieser Verbrauch wird dann als eigentliche Messgröße verwendet.
- Alle unnötigen Hintergrundprozesse werden beendet, um die Systemlast zu minimieren.
- Alle Messungen finden möglichst innerhalb von 20 Minuten statt, um die Temperatur des Geräts konstant zu halten.
- Alle Messungen werden auf demselben Testgerät durchgeführt, um die Hardwarebedingungen konstant zu halten. Dabei handelt es sich um ein Samsung Galaxy S23 mit dem Prozessor Qualcomm Snapdragon 8 Gen 2 und 8 GB Arbeitsspeicher, betrieben unter Android 16 (One UI 8.5), mit einer Akkukapazität von 3900 mAh [2, 34].
- Für alle Messungen werden dieselben Sessions als Eingabe verwendet, sodass alle Metriken unter identischen Bedingungen erfasst werden.
- Alle Modelle erhalten den identischen Prompt. Dieser ist bewusst auf das größte Modell, Gemma 2 2B, feinjustiert, um einen Referenzwert zu erhalten.

- Unmittelbar vor der Messung wird das Testgerät für mindestens 20 Minuten nicht genutzt, damit es abkühlen kann.
- Während der gesamten Messung bleibt das Testgerät eingeschaltet und die Session-Detailansicht geöffnet, da der Benchmark sonst unterbrochen wird.
- Alle Messungen erfolgen mit deutscher Spracheinstellung, sodass sämtliche Eingaben und Ausgaben deutschsprachig sind.

Auch wenn alle Modelle denselben szenariospezifischen Prompt erhalten, kann die Anzahl der Eingabe-Tokens je Modell schwanken. Jedes Modell besitzt einen eigenen Tokenizer mit einer anderen Implementierung eines Algorithmus und anderen Vokabulargrößen, um Textsequenzen in numerische Repräsentationen (Tokens) zu zerlegen [30]. Die konkreten Verfahren der verwendeten Modelle sind in Abschnitt 3.2 und Abschnitt 3.3 beschrieben. Für die Messbedingungen bedeutet das Folgendes: Der Top-K-Wert bleibt unbeeinflusst und ist von der Zerlegung selbst nicht betroffen. Die Kontextgröße von 1280 ist jedoch eine Größe, die je nach Tokenizer schneller (beispielsweise durch ein kleineres Vokabular) oder langsamer erreicht werden kann (Abschnitt 7.2).

5 Implementierung

Da die bestehende Anwendung bereits in Java umgesetzt ist und die Erweiterung auf sie aufbaut, wird sie ebenfalls in derselben Programmiersprache realisiert. Android Studio wird dabei als Entwicklungsumgebung verwendet, und der Build erfolgt über das Build-System Gradle. Die im Folgenden verwendeten Bibliotheken werden in den jeweiligen Abschnitten genannt. Der vollständige Quellcode der Anwendung liegt dem digitalen Anhang bei.

5.1 Architektur

Die Implementierung der Anwendung besteht aus den farblich gekennzeichneten Komponenten (Blau und Grün), die in Abbildung 5.1 dargestellt sind:

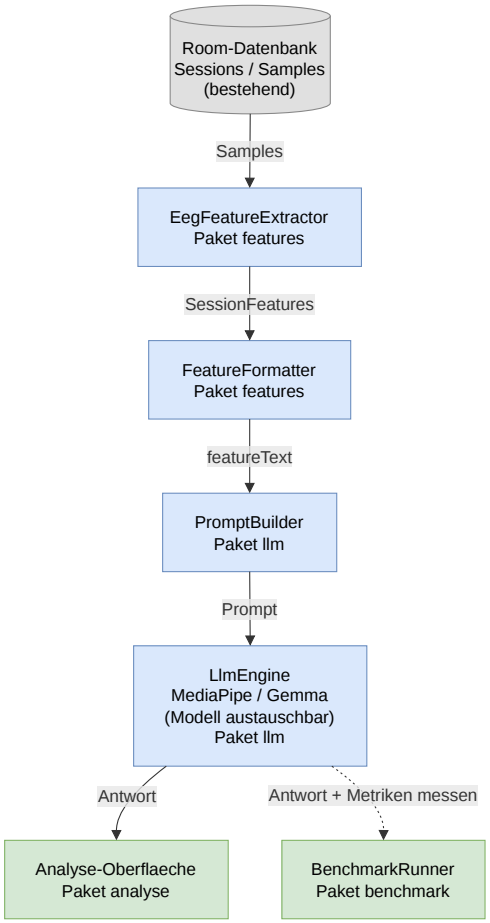


Abbildung 5.1: Architektur und Datenfluss der Anwendung. Blau und Grün kennzeichnen die neuen Komponenten, der Weg führt von der Datenbank bis zur Modellantwort (erstellt mit draw.io).

Der Weg startet über die Datenbank, aus der die Samples abgerufen werden:

- **EegFeatureExtractor** (`features`)**:** Hier werden die Samples verarbeitet, indem Merkmale berechnet werden.
- **FeatureFormatter** (`features`)**:** Wandelt die Merkmale in ein für das Sprachmodell verständliches Format um.
- **PromptBuilder** (`llm`)**:** Baut die Anweisung für den Prompt (je Szenario).
- **LlmEngine** (`llm`)**:** Führt das Modell (Gemma via MediaPipe) aus und gibt die Antwort zurück.
- **Analyse-Oberfläche** (`analyse`)**:** Zeigt letztlich die Antwort an.
- **Benchmark** (`benchmark`)**:** Misst die Verarbeitung (Laufzeit, Speicherverbrauch etc.).

Durch die Trennung dieser Pakete bleibt der Code übersichtlich und erweiterbar.

Ein Hauptmerkmal der Architektur ist die Kapselung des Sprachmodells in der Klasse `LlmEngine`. Die Klasse stellt eine einfache Schnittstelle bereit, die für das Laden, Anfragen und Freigeben verantwortlich ist. Dadurch ist die Modell- und Laufzeitumgebung vom Rest der Anwendung unbeeinflusst. So lassen sich Modelle einfach austauschen, ohne andere Komponenten anzupassen. Die drei Modelle Gemma 3 1B, Gemma 2 2B und Qwen 2.5 1.5B werden in dieser Arbeit allein durch den Wechsel des Modellpfads geladen. Aus diesem Grund würde sich an derselben Schnittstelle auch ein anderes Modell anbinden lassen. Möglich wäre es auch, eine andere Laufzeitumgebung oder gegebenenfalls ein systemintegriertes Modell anzubinden.

Das Gleiche lässt sich mit dem `PromptBuilder` erreichen. Dort lassen sich neue Szenarien ergänzen, ohne die restliche Anwendung anpassen zu müssen.

Auch mit dem `FeatureFormatter` ist eine einfache Änderung des Textes umsetzbar, da die Textdarstellung von der Berechnungslogik aller Merkmale entkoppelt ist.

Die Erweiterung verändert die Infrastruktur der bestehenden Anwendung nicht, sondern baut auf ihr auf. So wird beispielsweise die bestehende Datenbank verwendet und Neuerungen implementiert, ohne die vorhandene Live-Chart-Verarbeitung zu verändern. Zudem werden die Sessions am Anfang auf ihre Verwertbarkeit validiert. Neue Ansichten fügen sich in die bestehenden Konzepte der Anwendung ein. Neben dem hellen Modus unterstützt die neue Implementierung den Dunkelmodus sowie die Sprachen Deutsch und Englisch.

Der Benchmark verwendet denselben Analyseablauf wie eine vom Nutzer ausgelöste Einzelanalyse.

Zusätzlich zum Architekturdiagramm zeigt die folgende Abbildung 5.2 den zeitlichen Ablauf einer Analyse. Dadurch ist das Zusammenspiel der Komponenten zur Laufzeit besser nachvollziehbar.

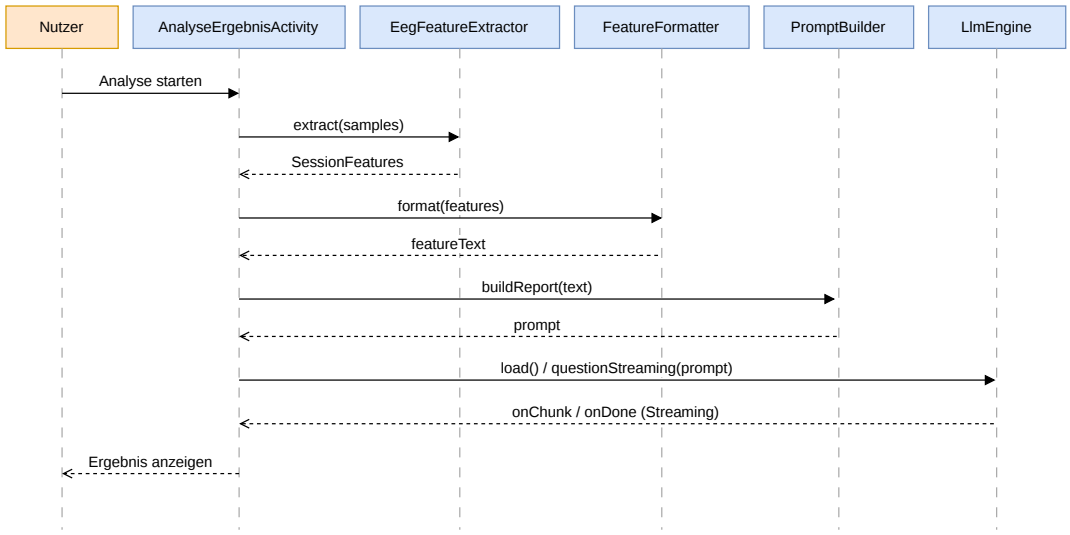


Abbildung 5.2: Sequenzdiagramm des Analyse-Ablaufs. Es zeigt das Zusammenspiel der Komponenten von der Auswahl im Dialog bis zur streamenden Ausgabe (erstellt mit draw.io).

Nun werden die einzelnen Komponenten der Architektur im Detail erläutert, beginnend mit der Merkmalsextraktion.

5.2 Implementierung der Merkmalsextraktion

Die Merkmalsextraktion ist im Paket `features` implementiert; zuständig dafür ist die Klasse `EegFeatureExtractor`. Sie durchläuft für jede Session eine feste Abfolge von Schritten: Als Erstes wird die Session aus der Datenbank geladen, daraufhin wird sie auf ihre Verwertbarkeit geprüft. Anschließend wird sie in gleich große Segmente aufgeteilt, bei denen dann jeweils die spektralen sowie statistischen Merkmale berechnet werden. Die Ergebnisse werden alle im Objekt `SessionFeatures` gesammelt und schließlich vom `FeatureFormatter` in einen lesbaren Text umgewandelt, den das Sprachmodell verarbeiten kann. Im Folgenden wird chronologisch auf die einzelnen Schritte genauer eingegangen.

5.2.1 Vorbereitung

Sobald die Klasse erzeugt wird, werden vorab alle wiederverwendbaren Bestandteile angelegt. Dazu gehört das Hann-Fenster, das für die Fourier-Transformation vorab berechnet wird, um bei jedem Fenster nicht erneut erzeugt zu werden. Außerdem wird das FFT-Objekt der Bibliothek JTransforms für die gewählte Fenstergröße instanziiert [37]. Der Standardwert für die Fenstergröße beträgt 256, kann aber variabel angepasst werden. Aus ihr und der Anzahl der Segmente wird die Mindestlänge `minSamples` der Session berechnet.

5.2.2 Validierung

Die Methode `validate` prüft eine Session auf ihre Verwertbarkeit. Es gibt zwei Kriterien, die erfüllt sein müssen. Zum einen muss die Session mindestens so viele Samples enthalten, wie in `minSamples` berechnet wurde, ansonsten wird `TOO_SHORT` zurückgegeben. Zum anderen muss der Anteil ungültiger Samples unter 50 % liegen. Ist dies nicht der Fall, wird `TOO_MANY_INVALID` zurückgegeben. Sind beide Kriterien erfüllt, wird die Analyse mit der Rückgabe von `OK` fortgesetzt, ansonsten wird dem Nutzer eine entsprechende Meldung angezeigt.

5.2.3 Segmentierung

Die Methode `extract` stellt den Einstiegspunkt der Merkmalsextraktion dar. Dort wird die Session in `SEGMENT_COUNT` gleich große Segmente (hier vier) zerlegt. Für jedes dieser Segmente werden separat die Merkmale (Listing 5.1) berechnet. Die Aufteilung in gleich große Stücke hilft dabei, den zeitlichen Verlauf innerhalb einer Session grob abzubilden, anstatt sie nur als ein einziges Mittel zu betrachten. Die einzelnen Segmente werden jeweils im `SessionFeatures`-Objekt abgelegt.

```
public SessionFeatures extract(List<SampleEntity> samples) {
    SessionFeatures features = new SessionFeatures();
    features.sessionId       = samples.get(0).sessionId;
    features.sampleCount     = samples.size();
    features.durationSeconds = samples.get(samples.size() - 1)
        .sampleCounterSeconds;
    features.segments        = new ArrayList<>();

    int segLength = samples.size() / SEGMENT_COUNT;
    for (int i = 0; i < SEGMENT_COUNT; i++) {
        int start = i * segLength;
        int end   = (i == SEGMENT_COUNT - 1) ? samples.size() : start
            + segLength;
        List<SampleEntity> slice = samples.subList(start, end);
        features.segments.add(buildSegment(slice));
        // Merkmale pro Segment berechnen
    }
    return features;
}
```

Listing 5.1: Zerlegung der Session in Segmente. Das letzte Segment nimmt die verbleibenden Samples auf, damit keine Messwerte verloren gehen.

Die Anzahl der Samples ist nicht immer genau durch `SEGMENT_COUNT` teilbar. Daher werden am Ende alle restlichen Samples in das letzte Segment aufgenommen. Dadurch gehen keine Messwerte verloren. `buildSegment` übernimmt die eigentliche Merkmalsextraktion je Segment.

In `buildSegment` werden zunächst Start- und Endzeit aus dem jeweils ersten sowie dem letzten Sample entnommen. Diese sind später für die aufbereiteten Zeitangaben je Segment wichtig. Anschließend werden die drei Merkmale berechnet. Dazu zählen die Band-Power je Kanal als spektrales Merkmal sowie die Spannweite je Kanal und der Anteil ungültiger Messwerte als statistische Merkmale. Die gesamten Werte werden, nachdem sie in einem Segment-Objekt gesammelt wurden, an `extract` zurückgegeben.

5.2.4 Spektrale Merkmale (Welch-Methode)

Für jeden der acht EEG-Kanäle wird die Leistung in den Frequenzbändern Delta (δ), Theta (θ), Alpha (α), Beta (β) und Gamma (γ) berechnet. Das geschieht nach dem Verfahren der Welch-Methode (Listing 5.2), bei der die Fourier-Transformation auf 50 % überlappende Segmente des Signals angewendet wird. Für jedes Fenster wird das Leistungsspektrum berechnet und in Bänder einsortiert, und über alle Fenster hinweg wird gemittelt. Durch die Mittelung wird ein stabileres Ergebnis geliefert, das weniger anfällig für zufällige Schwankungen ist.

```
private double[][] computeBandPower(List<SampleEntity> slice) {
    double[][] result = new double[CHANNELS][NUM_BANDS];
    int hop = windowSize / 2;
    // 50% Ueberlappung

    for (int i = 0; i < CHANNELS; i++) {
        double[] signal = extractChannelSignal(slice, i);
        double[] bandAccum = new double[NUM_BANDS];
        int windowCount = 0;

        for (int start = 0; start + windowSize <= signal.length;
        start += hop) {
            double[] window = Arrays.copyOfRange(signal, start, start +
            windowSize);
            double[] power  = powerSpectrum(window);
            // Fenster -> Spektrum
            double[] bands  = sumBands(power);
            // Spektrum -> Baender
            for (int b = 0; b < NUM_BANDS; b++) bandAccum[b] +=
            bands[b];
            windowCount++;
        }
        if (windowCount > 0) {
            for (int b = 0; b < NUM_BANDS; b++) {
                result[i][b] = bandAccum[b] / windowCount;
                // ueber alle Fenster mitteln (Welch)
            }
        }
    }
    return result;
}
```

Listing 5.2: Band-Power je Kanal nach dem Welch-Verfahren. Das Signal wird in überlappende Fenster zerlegt, in Bänder einsortiert und gemittelt.

Wichtig ist, dass die Band-Power vorerst je Kanal berechnet wird, die am Ende für die Aufbereitung für das Sprachmodell über alle acht Kanäle zusammengefasst wird. `windowCount` fungiert als Zähler, der sicherstellt, dass nur vollständig berechnete Fenster in die Mittelung eingehen.

Da jedes Sample des Unicorn Hybrid Black alle acht Kanäle gleichzeitig beinhaltet, wird in der Methode `extractChannelSignal` das Signal eines einzelnen Kanals (z. B. Kanal 1) über alle Samples eines Segments gesammelt. Dadurch entsteht ein durchgehender Signalverlauf dieses Kanals, auf den dann die Fensterung angewendet wird.

Die Methode `powerSpectrum` berechnet das Leistungsspektrum eines Fensters (Listing 5.3). Zunächst wird der Mittelwert (Gleichanteil) abgezogen. Danach wird das Hann-Fenster angewendet, um anschließend die Fourier-Transformation durchzuführen. Schließlich wird die Leistung je Frequenz-Bin berechnet, indem die Real- und Imaginärteile quadriert und summiert werden.

```
private double[] powerSpectrum(double[] window) {
    // 1. Mittelwert (Gleichanteil)
    double mean = 0;
    for (double v : window) mean += v;
    mean /= window.length;

    // 2. Mittelwert abziehen + Hann anwenden
    double[] data = new double[windowSize];
    for (int n = 0; n < windowSize; n++) {
        data[n] = (window[n] - mean) * hann[n];
    }

    // 3. FFT (in-place, JTransforms)
    fft.realForward(data);

    // 4. Leistung pro Frequenz-Bin
    double[] power = new double[windowSize / 2];
    power[0] = data[0] * data[0];
    // DC-Bin
    for (int k = 1; k < windowSize / 2; k++) {
        double re = data[2 * k], im = data[2 * k + 1];
        power[k] = re * re + im * im;
    }
    return power;
}
```

Listing 5.3: Berechnung des Leistungsspektrums eines Fensters. Vor der Transformation werden der Gleichanteil entfernt und das Hann-Fenster angewendet.

Die eigentliche Fourier-Transformation (`fft.realForward`) wird dabei durch die Bibliothek JTransforms bereitgestellt und nicht selbst implementiert [37]. Sie bietet eine Java-basierte Implementierung der Fourier-Transformation, die den Aufruf `realForward` für reelle Eingangssignale bereitstellt. Das Ergebnis wird dabei in dasselbe Array geschrieben, sodass die geraden Indizes die Real- und die ungeraden Indizes die Imaginärteile der Frequenz-Bins enthalten.

5.2.5 Zuordnung zu den Frequenzbändern

Das Spektrum liefert die Leistung je Frequenz-Bin. Diese müssen noch den Frequenzbändern zugeordnet werden (Listing 5.4). Dazu wird jeder Bin-Index über die Beziehung $\text{freq} = i \cdot f_s / N$ in eine Frequenz umgerechnet. Der Faktor f_s / N entspricht dabei der Frequenzauflösung Δf (Gleichung 2.2), also dem Frequenzabstand zwischen zwei benachbarten Bins. Dadurch können für jedes Frequenzband die entsprechenden Bins summiert werden.

```
private double[] sumBands(double[] power) {
    double[] bandSum = new double[NUM_BANDS];
    for (int i = 1; i < power.length; i++) {
        double freq = (double) (i * SAMPLE_RATE) / windowSize;
        // Bin-Index -> Frequenz
        for (FrequencyBand band : FrequencyBand.values()) {
            if (freq >= band.lowHz && freq < band.highHz) {
                bandSum[band.ordinal()] += power[i];
            }
        }
    }
    return bandSum;
}
```

Listing 5.4: Zuordnung der Frequenz-Bins zu den Bändern. Jeder Bin-Index wird in eine Frequenz umgerechnet und dem passenden Band zugeordnet.

Die Einteilung in die Frequenzbänder ist zentral im Aufzählungstyp (`enum`) `FrequencyBand` definiert, in dem jedes Band mit seiner unteren und oberen Grenzfrequenz hinterlegt ist. Die verwendeten Grenzen entsprechen den in Tabelle 2.1 genannten Bereichen.

5.2.6 Statistische Merkmale

Als erstes der zwei statistischen Merkmale wird der Peak-to-Peak-Wert berechnet. Dieser stellt die Spannweite je Kanal dar. Er beschreibt die Differenz zwischen dem höchsten und dem niedrigsten Messwert. Dadurch sind starke Ausschläge oder Drift sichtbar. Der zweite statistische Wert ist der Anteil der ungültigen Samples (`valid == 0`) an der Gesamtzahl im Segment. Er dient als Maß für die Vollständigkeit der Übertragung.

Bei der Berechnung des Peak-to-Peak-Werts wird das Segment einmal durchlaufen. Dabei werden alle acht Kanäle pro Sample verglichen, und das jeweils n-mal bis zum Ende eines Segments. Es merkt sich den niedrigsten sowie den höchsten Wert und kann anhand der Differenz aus beiden Werten die Spannweite berechnen. Bei dem Anteil der ungültigen Messwerte wird jedes Sample eines Segments auf sein Gültigkeitskennzeichen geprüft. Steht dieses auf 0, wird der Zähler an ungültigen Samples erhöht. Abschließend wird der Zähler durch die Gesamtzahl aller Samples dividiert. Dadurch erhält man den relativen Anteil an ungültigen Samples.

5.2.7 Ergebnis

Abschließend werden die berechneten Merkmale in einem Objekt `SessionFeatures` gesammelt. Dieses dient als Datencontainer, der alle Daten der Session sowie die Liste der Segmente mit ihren Merkmalen enthält. Das Ergebnis dieser Schritte ist die Grundlage für die anschließende Aufbereitung.

Das Objekt beinhaltet die Kennung der Session, ihre Dauer, die Anzahl der Samples sowie die Liste der Segmente. Jedes Segment besteht aus seiner Start- und Endzeit, der Band-Power der acht Kanäle in jeweils fünf Bändern, also 40 Werten je Segment, dazu den acht Spannweiten je Kanal und dem Anteil ungültiger Messwerte.

Die Merkmale liegen an dieser Stelle noch kanalweise und als reine Zahlenwerte vor. Erst der `FeatureFormatter` fasst sie über die Kanäle zusammen und wandelt sie in Text um.

5.2.8 Aufbereitung für das Sprachmodell

Die Aufbereitung erfolgt in der Klasse `FeatureFormatter`, bei der die Merkmale in ein lesbares Format für das Sprachmodell umgewandelt werden. Zunächst wird die absolute Band-Power über alle acht Kanäle summiert und anhand dieses Wertes in einen relativen Prozentwert je Band umgerechnet (Listing 5.5). Zusätzlich wird der Peak-to-Peak-Wert jedes Kanals pro Segment verglichen und der mit dem höchsten Ausschlag bestimmt.

```
private double[] relativeBandPower(SessionFeatures.Segment seg) {
    int channels = seg.bandPower.length;
    int bands    = seg.bandPower[0].length;

    double[] bandTotal = new double[bands];
    for (int c = 0; c < channels; c++) {
        for (int b = 0; b < bands; b++) {
            bandTotal[b] += seg.bandPower[c][b];
            // ueber alle Kanaele summieren
        }
    }
    double sumAll = 0;
    for (double v : bandTotal) sumAll += v;

    double[] rel = new double[bands];
    if (sumAll > 0) {
        for (int b = 0; b < bands; b++) {
            rel[b] = bandTotal[b] / sumAll * 100;
            // Anteil in Prozent
        }
    }
    return rel;
}
```

Listing 5.5: Umrechnung der absoluten Band-Power in relative Anteile. Die Werte werden über alle Kanäle summiert und als Prozentanteil ausgegeben.

Die Methode `format` erzeugt schließlich diesen aufbereiteten Text, der mit der szenariospezifischen Anweisung zu einem verwendbaren Prompt kombiniert wird (Abschnitt 5.4). Der Text wird Segment für Segment aufgebaut. Er beginnt mit einer Kopfzeile, die die Dauer der Session und die Anzahl der Kanäle nennt. Danach folgt für jedes Segment ein eigener Block mit dem Zeitraum, den fünf Bandanteilen, dem Anteil ungültiger Messwerte sowie dem Kanal mit dem größten Ausschlag samt seinem Wert in Mikrovolt. Die Bandanteile werden dabei auf ganze Prozent gerundet, der Anteil ungültiger Messwerte auf eine Nachkommastelle. Diese Rundung hält den Text kurz und vermeidet eine Genauigkeit, die für die Auswertung durch das Sprachmodell ohnehin keine Rolle spielt.

Im erzeugten Text sind die Segmente als Abschnitte bezeichnet. Ein Beispiel dafür zeigt Abbildung 5.3.

```
EEG-Session, Dauer 60.0s, 8 Kanäle.
Abschnitt 1 (0.0–15.0s):
    Delta 42%, Theta 18%, Alpha 21%, Beta 14%, Gamma 5%
    Ungültige Messwerte: 2.0% ungültig, größter Ausschlag Kanal 3 (128 µV)
Abschnitt 2 (15.0–30.0s):
    Delta 38%, Theta 20%, Alpha 24%, Beta 13%, Gamma 5%
    Ungültige Messwerte: 0.0% ungültig, größter Ausschlag Kanal 5 (96 µV)
Abschnitt 3 (30.0–45.0s):
    Delta 51%, Theta 16%, Alpha 18%, Beta 11%, Gamma 4%
    Ungültige Messwerte: 6.0% ungültig, größter Ausschlag Kanal 3 (210 µV)
Abschnitt 4 (45.0–60.0s):
    Delta 40%, Theta 19%, Alpha 22%, Beta 14%, Gamma 5%
    Ungültige Messwerte: 1.0% ungültig, größter Ausschlag Kanal 2 (105 µV)
```

Abbildung 5.3: Vom `FeatureFormatter` erzeugter Feature-Text. Je Segment stehen dort Bandanteile, ungültige Messwerte und der größte Ausschlag (Werte illustrativ).

5.3 Integration des Sprachmodells

Die gesamte Integration rund um das Sprachmodell ist im Paket `llm` implementiert. Dort befindet sich die Klasse `LlmEngine`, die für die gesamte Kommunikation mit dem Modell zuständig ist. Dabei hat die Klasse drei Kernaufgaben. Dazu zählen das Laden, Anfragen und Freigeben des Modells. Die MediaPipe LLM Inference API wird für die lokale Ausführung des Modells verwendet [15], die als Abhängigkeit `com.google.mediapipe:tasks-genai` eingebunden ist. Die quantisierten Modelle Gemma 3 1B, Gemma 2 2B und Qwen 2.5 1.5B liegen als `.task`-Dateien im Verzeichnis `/data/local/tmp/llm/` auf dem lokalen Gerät, wohin sie vorab per `adb` übertragen wurden. Dabei ist die Trennung der Modelle von der Anwendung bewusst gewählt. Die Anwendung enthält die Modelle somit nicht selbst, sie müssen vor der ersten Analyse auf das Gerät übertragen werden. Die Dateigröße der Modelle würde den Speicher der gesamten Applikation unnötig erhöhen. In der verwendeten 8-Bit-Quantisierung belegen die `.task`-Dateien rund 1 GB (Gemma 3 1B), 1,6 GB (Qwen 2.5 1.5B) beziehungsweise 2,6 GB (Gemma 2 2B) [25–27]. Im Weiteren wird der Lebenszyklus eines Modells in seiner Ausführungsreihenfolge beschrieben.

5.3.1 Modellauswahl und Pfade

Die jeweils verwendeten Modelle sind als konstante Pfade hinterlegt. Diese werden mithilfe der Methode `pathFor` anhand einer kurzen Kennung wie `1b`, `2b` oder `qwen` selektiert. Falls die Kennung ungültig ist, wird automatisch das kleinste Modell ausgewählt. Die Auswahl des Modells geschieht zuvor im Analyse-Dialog oder im Benchmark, wobei die Kennung an die `LlmEngine` übergeben wird.

5.3.2 Laden und Konfiguration

Das Laden des Modells besteht aus dem Konfigurieren der Optionen (Listing 5.6). Hierzu werden in einem `LlmInferenceOptions`-Objekt die Pfadangabe, `maxTopK` und `maxTokens` festgelegt. Die Obergrenze der wählbaren Token wird auf 64 gesetzt. Als maximale Anzahl

der Token wird 1280 festgelegt. Hierbei gelten diese Werte über alle Modelle hinweg, um einen fairen Vergleich zu bieten (Kapitel 4). Erzeugt und einsatzbereit ist das Modell anschließend nach dem Aufruf von `createFromOptions`.

```
public void load(Context context) {
    LlmInferenceOptions options = LlmInferenceOptions.builder()
            .setModelPath(modelPath)
            .setMaxTopK(64)
            // Sampling-Obergrenze
            .setMaxTokens(1280)
            // Kontextgroesse (Prompt + Ausgabe)
            .build();

    llm = LlmInference.createFromOptions(context, options);
}
```

Listing 5.6: Laden und Konfigurieren des Modells. Kontextgröße und Sampling-Obergrenze sind für alle Modelle identisch gesetzt.

5.3.3 Streamende Ausgabe

Die Antwort eines Modells erfolgt in einem Stream, bei dem die Token nach und nach geliefert werden (Listing 5.7). Der vom `PromptBuilder` erzeugte Prompt wird über `generateResponseAsync` an das Modell übergeben. Dabei wird jedes Teilstück über einen Callback gemeldet. So wird der Verlauf der Entstehung der Antwort in Echtzeit an die Benutzeroberfläche weitergeleitet. Gleichzeitig wird der Time-To-First-Token gemessen. Dieser wird mit der vollständigen Antwort samt Gesamtdauer zurückgegeben.

```
public void questionStreaming(String prompt, StreamCallback callback) {
    StringBuilder full = new StringBuilder();
    long start = System.nanoTime();
    long[] ttft = {-1};

    llm.generateResponseAsync(prompt, (partial, done) -> {
        if (ttft[0] < 0) {
            ttft[0] = (System.nanoTime() - start) / 1_000_000;
            // erstes Token -> TTFT festhalten
        }
        full.append(partial);
        callback.onChunk(partial);
        // Teilstueck weiterreichen
        if (done) {
            long totalMs = (System.nanoTime() - start) / 1_000_000;
            callback.onDone(full.toString(), ttft[0], totalMs);
        }
    });
}
```

Listing 5.7: Streamende Ausgabe mit Messung der Latenz. Beim ersten Teilstück wird die TTFT festgehalten, am Ende zusätzlich die Gesamtdauer.

Abgesehen von der streamenden Ausgabe gibt es noch eine einfache Anfrage, die auf die vollständige Antwort wartet und sie am Stück zurückgibt. Diese Anfrage wird im Benchmark für die Aufwärmmessung genutzt, da weder eine Live-Anzeige noch die Latenzwerte benötigt werden.

5.3.4 Zählen der Token

Die `LlmEngine` stellt die Methode `tokenCount` bereit, die die Anzahl der Tokens berechnet. Diese sind für die Metriken der Evaluation wichtig. Dabei ruft `tokenCount` die Methode `sizeInTokens` der MediaPipe-Schnittstelle auf. Gezählt wird immer anhand des Tokenizers, den das jeweils verwendete Modell bereitstellt. Diese Logik wird einmal bei dem Prompt selbst verwendet, woraus sich die Anzahl der Eingabe-Tokens ergibt, und bei der anschließenden Ausgabe des Modells, woraus zusammen mit der Inferenzdauer die Tokens pro Sekunde berechnet werden. Der Aufruf erfolgt dabei jeweils außerhalb des Callbacks der MediaPipe-Schnittstelle.

5.3.5 Freigabe

Am Ende wird das Modell über `close` freigegeben. Dabei wird die MediaPipe-Instanz geschlossen und die Referenz zurückgesetzt. Das ist nötig, weil ein geladenes Modell

mehrere Gigabyte Arbeitsspeicher belegt, der sonst bis zum Beenden der Anwendung reserviert bliebe. In der Analyse geschieht das nach dem Ende der Ausgabe, im Benchmark nach jeder Konfiguration.

5.4 Prompt-Design und iterative Optimierung

Wie schon in Abschnitt 4.3 erwähnt, besteht der Prompt aus den aufbereiteten Merkmalen mit einer szenariospezifischen Anweisung. Dieser ist für alle Modelle identisch. Der Prompt wurde in der Klasse `PromptBuilder` realisiert. Der Weg bis zum finalen Prompt war iterativ: Er wurde anhand der Modellausgaben verfeinert und anschließend für die Evaluation final festgehalten.

Bei der Optimierung des Prompts ergaben sich folgende Design-Vorgaben, die jeweils auf ein beobachtetes Problem der frühen Ausgaben zurückgehen:

- **Datentreue:** Dem Modell wird klargestellt, nur die genannten Werte zu verwenden. Dies soll verhindern, dass das Modell selbst neue Annahmen trifft. In früheren Ausgaben wurden fälschlicherweise Begriffe wie „Dominanzindex“ verwendet.
- **Reines Textformat:** Die Ausgabe des Modells soll textuelle Informationen erzeugen. Es wird vorgegeben, dass Sonderzeichen und Markdown verboten werden. Zuvor entstanden Formatierungszeichen wie `**` in den Ausgaben.
- **Eindeutige Merkmalsbenennung:** Im `FeatureFormatter` wird der Anteil ungültiger Werte als „Ungültige Messwerte: X %“ statt als „Qualität: X %“ ausgegeben. Zuvor interpretierte das Modell „Qualität: X %“ fälschlich als schlechte Qualität.
- **Begrenzung der Interpretation:** Der Prompt untersagt falsche Interpretationen wie zum Beispiel Deutungen in Richtung medizinischer Diagnosen. Solche Fälle haben davor ein Problem dargestellt.
- **Kompaktheit:** Wenige Sätze, keine Begrüßung oder Rückfragen.

Nach der Optimierung wurde der finale Prompt eingefroren und bildete die Grundlage für alle Messungen und den Benchmark. Listing 5.8 zeigt die finalen Prompts je Szenario.

```
// Szenario 1: Session-Report
public String buildReport(String featureText) {
    return "Fasse die folgenden EEG-Messwerte in 3 bis 4 Saetzen
        sachlich zusammen. "
         + "Schreibe nur den Bericht: keine Begruessung, keine
             Rueckfragen, kein Schlusssatz. "
         + "Schreibe reinen Fliesstext ohne Sonderzeichen (kein
             Markdown). "
         + "Nenne das dominante Frequenzband, die Signalqualitaet und
             auffaellige Ausschlaege. "
         + "Erfinde nichts und nutze nur die folgenden Werte:\n\n"
         + featureText;
}
// Szenario 2: Artefakt-Erklaerung
public String buildArtefaktErklaerung(String featureText) {
    return "Analysiere die folgenden EEG-Messwerte und beziehe dich
        konkret auf die genannten Werte. "
         + "Benenne die auffaelligsten Punkte dieser Session (z. B.
             sehr hohe Ausschlaege, "
```

```
            + "ungueltige Daten oder ein stark dominantes Band) und
                erklaere kurz ihre moegliche "
            + "technische Ursache (z. B. Bewegung, Elektroden, Verbindung
                ). Stelle keine "
            + "medizinischen Diagnosen. Schreibe wenige Saetze als
                sachlichen Fliesstext, ohne "
            + "Begruessung und ohne Sonderzeichen (kein Markdown).
                Erfinde nichts und nutze nur "
            + "die folgenden Werte:\n\n"
            + featureText;
}
// Szenario 3: Session-Vergleich
public String buildVergleich(String featureTextA, String featureTextB)
     {
     return "Vergleiche die folgenden zwei EEG-Sessions und beschreibe
         die wichtigsten "
           + "Unterschiede in einfachen Worten (z. B. Banddominanz,
               Signalqualitaet, Ausschlaege). "
           + "Schreibe sachlichen Fliesstext ohne Sonderzeichen (kein
               Markdown). Erfinde keine "
           + "Kennzahlen oder Begriffe. Schreibe nur den Vergleich,
               keine Begruessung. "
           + "Nutze nur die folgenden Werte.\n\n"
           + "Session A:\n" + featureTextA
           + "\n\nSession B:\n" + featureTextB;
}
```

Listing 5.8: Aufbau der Prompts je Szenario in der finalen, eingefrorenen Fassung. Alle drei enden mit dem aufbereiteten Merkmalstext.

Je nachdem, welche Sprache in den App-Einstellungen ausgewählt wurde, werden die Ausgaben in Deutsch oder Englisch erzeugt. Die Übersetzung funktioniert anhand überladener Methoden im `PromptBuilder`, die die bestehenden Methoden aufrufen und im Falle der englischen Sprache einen entsprechenden Zusatz an den Prompt anhängen.

5.5 Benutzeroberfläche

Als Grundlage für das Auslösen der neuen Funktionen dient die Session-Detailseite, die es schon in der ursprünglichen Anwendung gab (Abbildung 5.4). Hinzugekommen sind zwei neue Knöpfe. Die Benutzeroberfläche, die für die KI-Analyse zuständig ist, befindet sich im Paket analyse. Die Oberfläche startet mit dem Auslösen des Analyseknopfes. Daraufhin erfolgt ein Dialog zum Einstellen der Parameter und anschließend die Anzeige der Ergebnisse. Zudem gibt es einen Benchmark-Knopf, der über einen weiteren Dialog die automatisierte Messreihe startet.

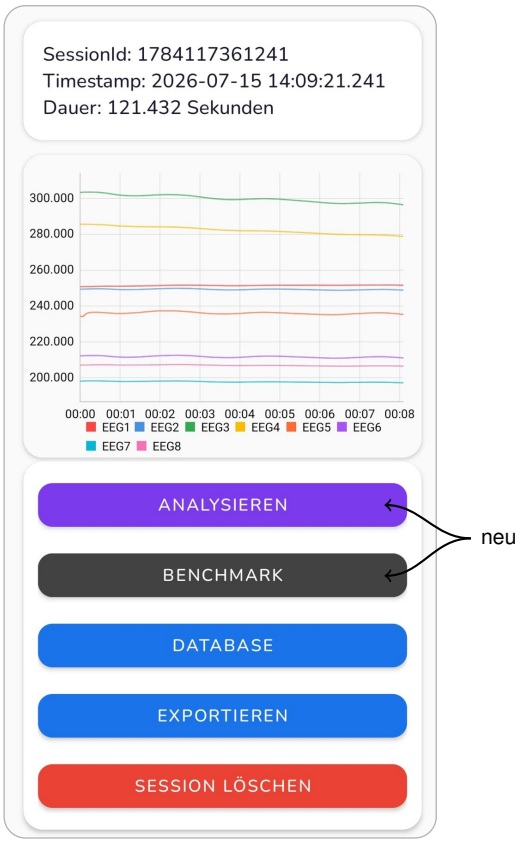


Abbildung 5.4: Session-Detailansicht als Einstieg in die neuen Funktionen. Die markierten Knöpfe starten die KI-Analyse und die automatisierte Messreihe.

5.5.1 Analyse-Dialog

Nachdem man auf „Analysieren“ drückt, öffnet sich ein Dialog, also ein kompaktes Fenster auf derselben Oberfläche, in dem der Nutzer die Parameter der Analyse festlegen kann. Dazu zählen die Szenarien (Report, Artefakt-Erklärung oder Vergleich) sowie die Sprachmodelle (Gemma 3 1B, Gemma 2 2B oder Qwen 2.5 1.5B). Für Entwickler gibt es zudem unten im Dialog weitere Optionen. So lassen sich zusätzlich die Fenstergröße einstellen und über einen Schalter Messmetriken aktivieren. Im Falle, dass das Szenario Vergleich gewählt wird, kann eine zusätzliche Session ausgewählt werden, die dann als Vergleich herangezogen wird. Der Dialog ist in der Klasse `AnalyseDialog` implementiert. Nachdem der Start-Knopf betätigt wird, werden die ausgewählten Parameter als Intent-Extras an die Klasse `AnalyseErgebnisActivity` übergeben, in der die eigentliche Analyse beginnt.

Abbildung 5.5 zeigt den beschriebenen Analyse-Dialog.

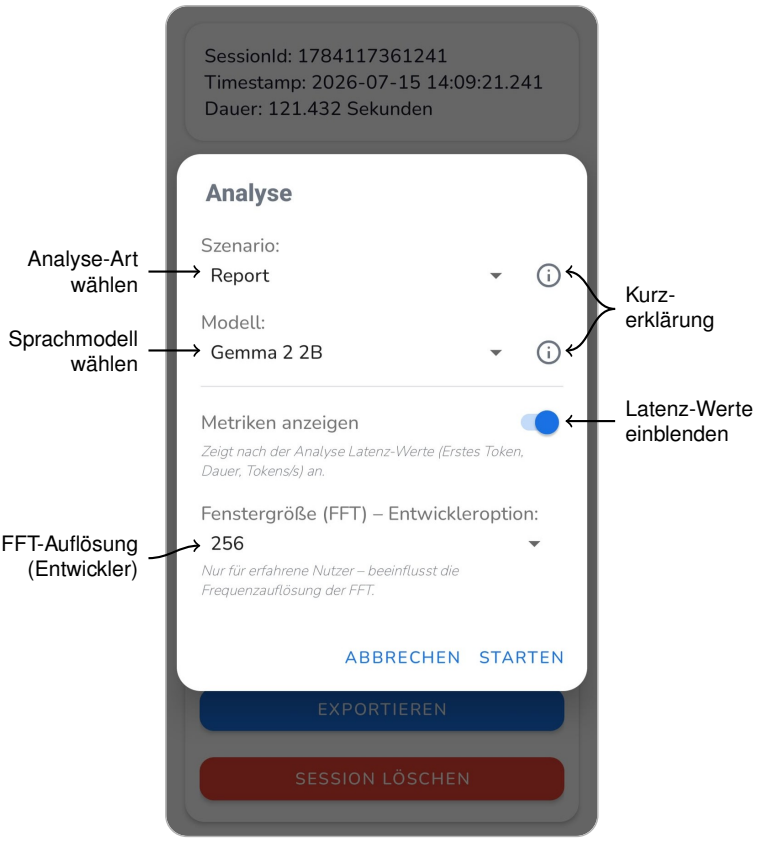


Abbildung 5.5: Analyse-Dialog zur Auswahl der Parameter. Neben Szenario und Sprachmodell stehen unten die Entwickleroptionen für Fenstergröße und Metriken.

5.5.2 Ergebnis-Ansicht

Die Logik der gesamten Ergebnis-Ansicht befindet sich in der Klasse `AnalyseErgebnisActivity`. Anhand der ausgewählten Parameter wird zunächst der Prompt erstellt. Dieser wird dann über die in Abschnitt 5.3 beschriebene `LlmEngine` an das Sprachmodell gestellt. Da die Ausgabe streamend verläuft, ist ein Interface `LlmEngine.StreamCallback` implementiert (Listing 5.9). Dort befinden sich die beiden Methoden `onChunk` und `onDone`. Für die Anzeige der Ausgabe ist die Methode `onChunk` zuständig. Diese sorgt für die fortlaufende Aktualisierung der Antwort vom Modell an die Ergebnis-Ansicht. Mit `onDone` wird das Ende der Ausgabe signalisiert, sodass die Metriken nachträglich angezeigt werden können.

```
@Override
public void onChunk(String partial) {
    runOnUiThread(() -> {
        progressBar.setVisibility(View.GONE);
        tvReport.append(partial);    // live anhaengen, nicht setText
    });
}
```

Listing 5.9: Live-Anzeige der streamenden Ausgabe. Jedes Teilstück wird angehängt statt gesetzt, damit der Text während der Erzeugung wächst.

Ein Modell kann mehrere Gigabyte groß sein, weshalb die Analyse in einem eigenen Thread läuft. Ansonsten könnte das Laden die Oberfläche blockieren. Änderungen an der Benutzeroberfläche können in Android nur im Haupt-Thread erfolgen, weshalb dafür `runOnUiThread` verwendet wird. Falls beim Vorbereiten der Daten oder beim Laden des Modells ein Fehler auftreten sollte, wird eine dementsprechende Fehlermeldung im Textbereich angezeigt, sodass kein Absturz erfolgt.

Die Ergebnis-Ansicht zeigt als Erstes eine Ladeanimation an. Währenddessen wird das Modell geladen. Sobald die Ausgabe beginnt, wächst der Text aufgrund der Streaming-Mechanik nach und nach. Ganz oben ist sichtbar, welches Modell und Szenario ausgewählt wurde. Wenn der Metrik-Schalter im Dialog aktiviert wird, sind zusätzlich die Latenz-Werte (erstes Token, Dauer und Tokens pro Sekunde) sichtbar. Sobald die Ausgabe vollständig ist, erscheint unten rechts ein Knopf zum Teilen der Analyse. Abbildung 5.6 zeigt die Ansicht.

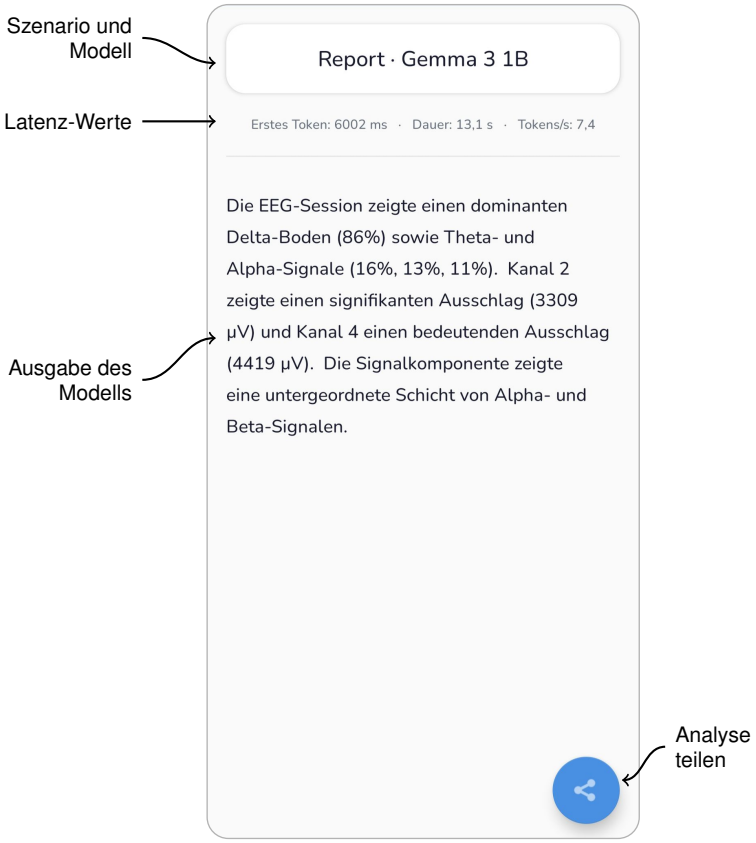


Abbildung 5.6: Ergebnis-Ansicht der KI-Analyse. Oben stehen Szenario, Modell und die optionalen Latenzwerte, darunter wächst die Ausgabe während der Erzeugung.

Der Teilen-Knopf erscheint, sobald die erzeugte Analyse abgeschlossen ist. Mit diesem kann man die Ausgabe als .txt-Datei teilen. Für diese Funktion wird ein `FileProvider` verwendet, der in der bestehenden Anwendung bereits für den Export der Messdaten eingerichtet ist. Beim Scrollen wird die Deckkraft des Knopfes auf 30 % reduziert, damit der Leser den kompletten Text unverdeckt lesen kann. Nach einer Sekunde ohne weitere Scrollbewegung erlischt der Effekt und die Deckkraft steigt wieder auf das Maximum.

Zusätzlich wird geprüft, ob eine Aufnahme die Mindestlänge erfüllt und die Schwelle der ungültigen Samples nicht überschreitet. Ist dies nicht der Fall, wird die Analyse abgebrochen und eine entsprechende Meldung angezeigt (Abschnitt 5.2).

5.5.3 Benchmark-Dialog

Neben dem Analyse-Knopf gibt es in derselben Ansicht noch einen Benchmark-Knopf, über den man die automatisierte Messreihe starten kann. Auch dort startet zuvor ein Dialog (Abbildung 5.7), mit dem man für das Vergleichsszenario eine Session auswählen muss.

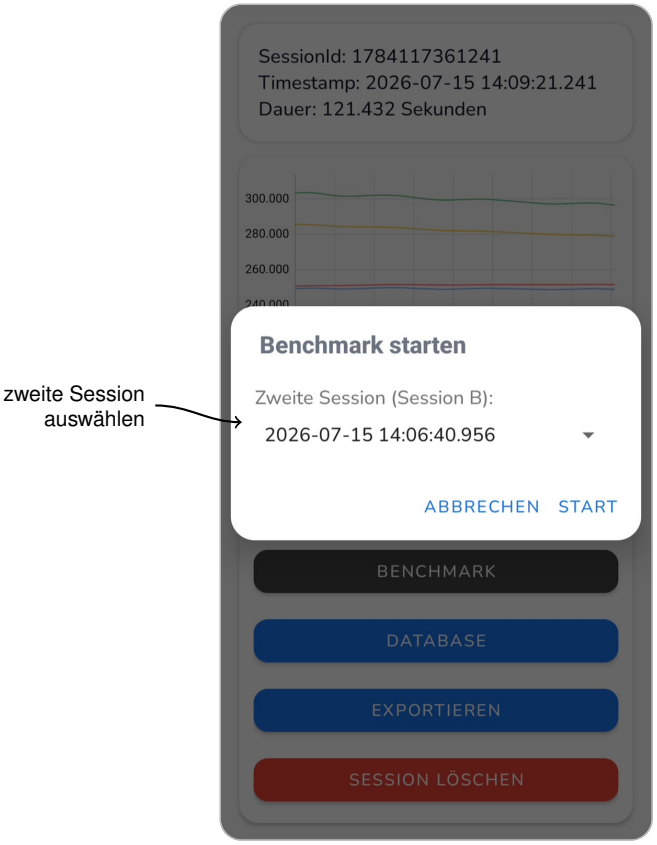


Abbildung 5.7: Benchmark-Dialog vor dem Start der Messreihe. Hier wird die zweite Session festgelegt, die im Vergleichsszenario genutzt wird.

Zuständig für den Benchmark-Dialog ist die Methode `showBenchmarkDialog`. Diese befindet sich in keiner eigenen Klasse, sondern in `SessionDetailActivity`, welche schon in der bestehenden Anwendung vorhanden war. Die restliche Benchmark-Logik ist in Abschnitt 5.6 beschrieben.

5.6 Benchmark

Die gesamte Logik des Benchmarks befindet sich im Paket `benchmark`. Von dort aus werden alle Messungen, die in Abschnitt 4.4 definiert sind, automatisiert durchgeführt. Dabei werden alle Kombinationen durchlaufen und in die CSV-Datei geschrieben. Die Implementierung wird in drei Klassen aufgeteilt: `BenchmarkRunner` steuert den Ablauf, `Metrics` liefert die Messwerte für Speicher und Akku, und `BenchmarkLogger` schreibt die Ergebnisse. Den Start bildet der Benchmark-Button in der Session-Detailansicht. Dieser Ablauf ist in Abbildung 5.8 dargestellt.

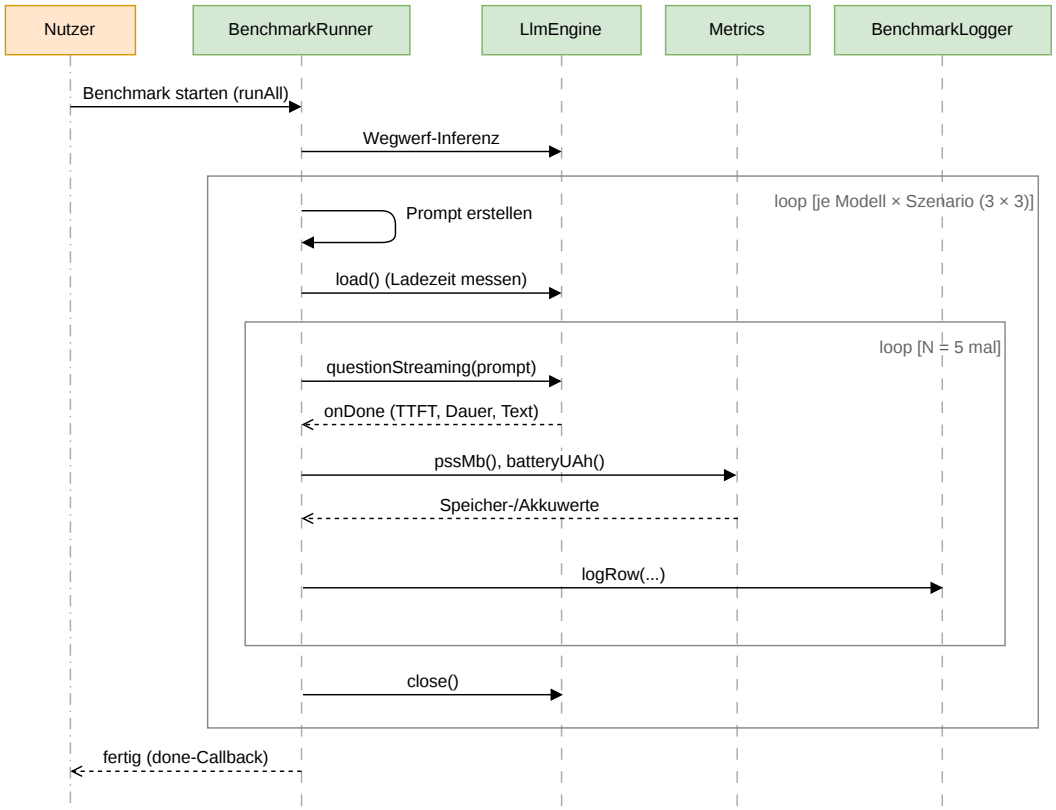


Abbildung 5.8: Sequenzdiagramm des Benchmark-Ablaufs. Es zeigt das Zusammenspiel von Steuerung, Messwerterfassung und Protokollierung (erstellt mit draw.io).

Die einzelnen Schritte dieses Ablaufs werden in den folgenden Abschnitten anhand des Codes näher erläutert.

5.6.1 Ablauf

Der Hintergrund-Thread `runAll` ist für den gesamten Messlauf verantwortlich (Listing 5.10). Dieser sorgt dafür, dass die Oberfläche nicht blockiert wird. Zum Start wird eine einzelne Wegwerf-Messung durchgeführt, um einen Kaltstart zu vermeiden. Danach startet die eigentliche Messung über alle Kombinationen. Ist der gesamte Lauf beendet, wird der übergebene `done`-Callback aufgerufen, über den die Oberfläche informiert wird.

```
public void runAll(long sessionA, long sessionB, int numberConfig,
Runnable done) {
    new Thread(() -> {
        String[] modelle   = {"1b", "2b", "qwen"};
        String[] szenarien = {"report", "artefakt", "vergleich"};

        // Aufwaermphase: eine Wegwerf-Inferenz gegen Kaltstart-Effekte
        try {
            LlmEngine warm = new LlmEngine(LlmEngine.
            pathFor(modelle[0]));
            warm.load(context);
            warm.question("Test");
            warm.close();
        } catch (Exception ignored) {}

        for (String modell : modelle) {
            for (String szenario : szenarien) {
                runConfig(modell, szenario, sessionA, sessionB,
                numberConfig);
            }
        }
        if (done != null) done.run();
        // Benchmark fertig -> UI benachrichtigen
    }).start();
}
```

Listing 5.10: Ablauf des Benchmarks mit Aufwaermphase. Vor der eigentlichen Messreihe wird eine Wegwerf-Inferenz ausgefuehrt.

5.6.2 Messung je Konfiguration

Für jede Kombination wird `runConfig` einmal aufgerufen. Dabei wird das Modell geladen und die Ladezeit gemessen, bevor dieselbe Anfrage n = 5 mal wiederholt wird. `questionStreaming` ist für die asynchrone Verarbeitung zuständig. Diese wird mithilfe eines `CountDownLatch` synchronisiert, der das Warten bei jeder Wiederholung übernimmt. Die Schleife wartet bei `await`, bis der `onDone`-Callback per `countDown` das Ende der Inferenz meldet. Erst danach werden die Speicher- und Akkuwerte ausgelesen und als eine Zeile gespeichert. Bei einem Fehler wird dies in der CSV-Datei gekennzeichnet.

Die Prompts werden immer vor der Wiederholungsschleife erzeugt. Das Modell wird einmal geladen und dabei die Ladezeit sowie die Anzahl der Eingabe-Tokens bestimmt. Daher bleiben beide Werte innerhalb einer Kombination aus Szenario und Modell über alle fünf Messungen identisch. Der Benchmark verwendet dabei immer die Standardfenstergröße von 256 Samples, unabhängig davon, was im Analyse-Dialog eingestellt ist.

Die beschriebene Wiederholung mit der Synchronisation über den `CountDownLatch` ist in Listing 5.11 dargestellt.

```
for (int i = 0; i < nc; i++) {
    long[]   ttftHolder  = {0};
    long[]   totalHolder = {0};
    String[] textHolder  = {""};
    CountDownLatch latch = new CountDownLatch(1);

    engine.questionStreaming(prompt, new LlmEngine.StreamCallback() {
        @Override public void onChunk(String partial) {
            /* im Benchmark egal */ }
        @Override public void onDone(String fullText, long ttftMs,
        long totalMs) {
            ttftHolder[0]  = ttftMs;
            totalHolder[0] = totalMs;
            textHolder[0]  = fullText;
            latch.countDown();          // Riegel oeffnen
        }
    });

    latch.await();                      // warten, bis onDone kam

    long inferMs = totalHolder[0];
    int outputTokens = engine.tokenCount(textHolder[0]);
    double tokensPerSec = outputTokens / (inferMs / 1000.0);

    long pssMb   = Metrics.pssMb();
    long battUAh = Metrics.batteryUAh(context);
    int  battPct = Metrics.batteryPct(context);

    logger.logRow(modell, szenario, promptTokens, outputTokens,
                  loadMs, inferMs, ttftHolder[0], tokensPerSec,
                  pssMb, battUAh, battPct, "ok");
}
```

Listing 5.11: Wiederholte Messung mit Synchronisation ueber CountDownLatch. Die Schleife wartet, bis die asynchrone Ausgabe abgeschlossen ist.

5.6.3 Erfassung der Ressourcen

Die Klasse `Metrics` ist eine Hilfsklasse, die die Speicher- und Akkuwerte liefert. Der Speicherverbrauch wird als PSS über `Debug.MemoryInfo` ausgelesen und von Kilobyte in Megabyte umgerechnet. Über den `BatteryManager` wird der aktuelle Ladezustand des Akkus in Prozent und Mikroamperestunden (µAh) ermittelt.

Der Speicherwert wird für den gesamten Prozess der Anwendung ermittelt. Dazu zählt nicht nur das geladene Modell selbst, sondern auch der übrige Speicherbedarf der App. Der Akkuwert beschreibt zudem den momentanen Akkuzustand und nicht den Verbrauch selbst. Dieser ergibt sich schließlich aus der Differenz zweier Akkuwerte über eine Messreihe hinweg (Abschnitt 4.4.1).

5.6.4 Protokollierung

Die Klasse `BenchmarkLogger` schreibt die Ergebnisse in die Datei `benchmark.csv`. Ist die Datei schon vorhanden, wird die bestehende gelöscht und eine neue erstellt. Sie enthält eine Kopfzeile mit allen Metriken und darunter die einzelnen Zeilen, die jeweils eine Messung darstellen. Die so entstehende CSV bildet die Datengrundlage für die Evaluation (Kapitel 6). Jede Zeile enthält einen Zeitstempel, das Modell, das Szenario, die Anzahl der

Eingabe- und Ausgabe-Tokens, die Ladezeit, die Inferenzdauer, die TTFT, die Tokens pro Sekunde, den Speicherwert sowie die Akkuwerte in Mikroamperestunden und Prozent. Abgeschlossen wird sie durch den Status der Messung.

6 Evaluation

Dieses Kapitel präsentiert die Durchführung (Abschnitt 4.4.2) sowie die Ergebnisse der Metriken (Abschnitt 4.4.1), um zu beurteilen, inwiefern sich die lokalen Modelle für die Verarbeitung von EEG-Daten eignen.

6.1 Durchführung der Messungen

Wie schon in Abschnitt 4.4 beschrieben, wird die Eingabe für alle Modelle identisch gehalten. Insgesamt werden so alle neun Kombinationen aus drei Modellen und drei Szenarien auf dem Samsung Galaxy S23 vermessen und anschließend in einer neu erstellten CSV-Datei protokolliert. Dies geschieht je fünfmal, wobei die erste Messung jeder Kombination verworfen wird, da sie durch das Initialisieren des Modells länger ausfallen kann (Abschnitt 4.4). In den Rohdaten ist dieser Effekt bei einzelnen Kombinationen deutlich sichtbar, in anderen tritt er dagegen nicht auf. Ein Beispiel ist Gemma 3 1B, denn dort unterscheidet sich im Report-Szenario die Gesamtdauer bei der ersten Messung mit 13,1 s gegenüber 6,7 s in den folgenden vier Läufen (Tabelle B.1). In die Mittelwerte fließen je Kombination die verbleibenden vier Messungen ein. Die rohen Messwerte liegen vollständig und ungemittelt in Tabelle B.1. Vor der ersten Messung wird ein Aufwärmlauf durchgeführt.

Als Eingabe für das Modell werden zwei bestimmte Sessions verwendet. Beide sind beim Tragen des Unicorn-Hybrid-Headsets durch den Verfasser entstanden und haben eine Laufzeit von knapp über 120 Sekunden. Im Report- und Artefakt-Szenario wird allein Session A übergeben (Abbildung A.1), im Vergleichsszenario zusätzlich Session B (Abbildung A.2).

6.2 Latenz

Im Folgenden werden die Ergebnisse der Latenzmessungen vorgestellt.

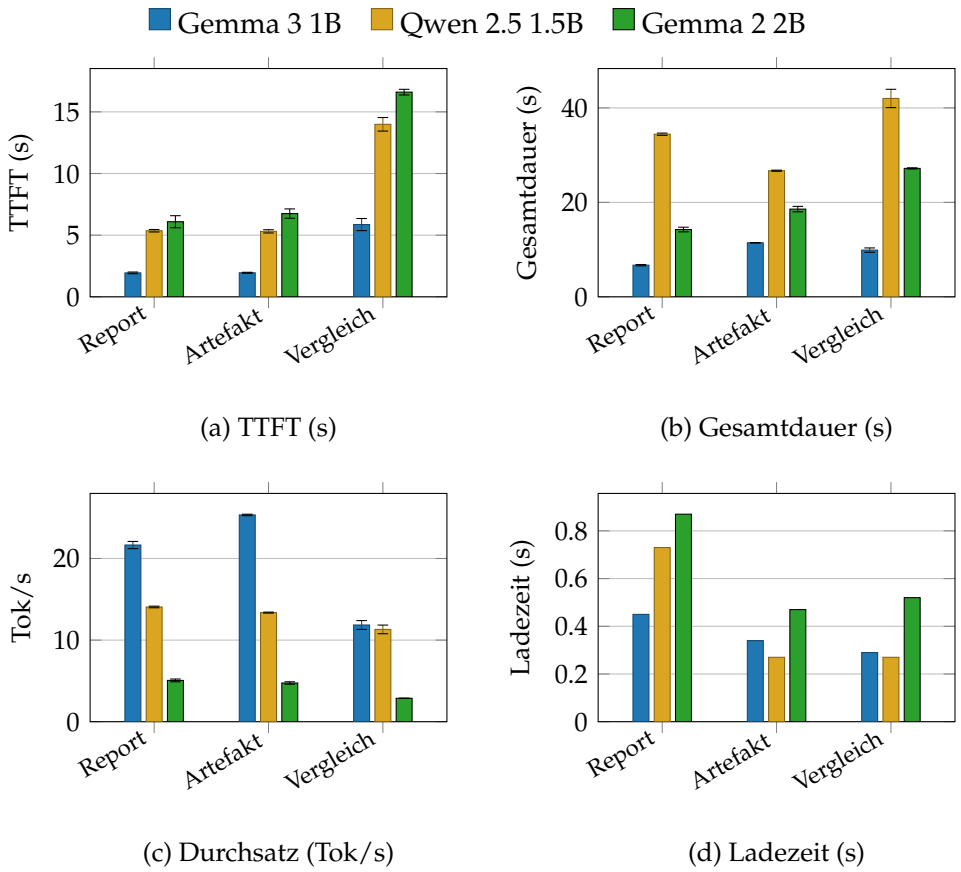
Tabelle 6.1: Latenzmetriken je Modell und Szenario (Mittel aus vier Wiederholungen ± Standardabweichung).

Modell	Szenario	Ladezeit (s)	TTFT (s)	Gesamtdauer (s)	Tok/s
Gemma 3 1B	Report	0,45	1,94 ± 0,07	6,70 ± 0,14	21,65 ± 0,44
	Artefakt	0,34	1,95 ± 0,04	11,44 ± 0,04	25,34 ± 0,09
	Vergleich	0,29	5,86 ± 0,49	9,89 ± 0,46	11,85 ± 0,54
Qwen 2.5 1.5B	Report	0,73	5,36 ± 0,10	34,43 ± 0,24	14,06 ± 0,10
	Artefakt	0,27	5,31 ± 0,13	26,70 ± 0,12	13,37 ± 0,06
	Vergleich	0,27	13,99 ± 0,55	42,00 ± 1,94	11,31 ± 0,53
Gemma 2 2B	Report	0,87	6,09 ± 0,49	14,25 ± 0,49	5,06 ± 0,17
	Artefakt	0,47	6,75 ± 0,38	18,57 ± 0,58	4,74 ± 0,15
	Vergleich	0,52	16,60 ± 0,23	27,19 ± 0,13	2,87 ± 0,01

Tabelle 6.1 zeigt, dass der **Durchsatz (Tok/s)** bei dem Modell Gemma 3 1B am höchsten ist (bis zu 25,34 Tok/s), während das Modell Gemma 2 2B den geringsten Durchsatz aufweist (minimal 2,87 Tok/s). Der Durchsatz ist dabei im Vergleichsszenario über alle Modelle hinweg am geringsten. Die **Ladezeit** ist bei Gemma 2 2B am höchsten; die beiden kleineren Modelle Gemma 3 1B und Qwen 2.5 1.5B laden schneller. Sie liegt durchweg unter einer Sekunde (zwischen 0,27 und 0,87 s). Die **TTFT** ist bei Gemma 3 1B am geringsten, während sie bei Gemma 2 2B am höchsten ist. Mit Ausnahme von Qwen 2.5 1.5B/Artefakt steigt die TTFT mit dem Szenarioniveau an und erreicht ihren Höchstwert bei Gemma 2 2B mit 16,60 s. Die **Gesamtdauer** reicht von etwa 6,70 bis 42,00 s und variiert stark zwischen den Modellen und Szenarien.

Die **Standardabweichungen** fallen überwiegend gering aus. Von den 27 erfassten Werten liegt bei 26 eine Streuung von unter eins vor, was auf eine stabile Messung hinweist. Lediglich bei Qwen 2.5 1.5B/Vergleich ist die Standardabweichung mit 1,94 s (Gesamtdauer) deutlich höher als bei allen anderen Messungen.

Zur besseren Vergleichbarkeit sind die Latenzmetriken in Abbildung 6.1 zusätzlich grafisch dargestellt.



(a) TTFT (s)

(b) Gesamtdauer (s)

(c) Durchsatz (Tok/s)

(d) Ladezeit (s)

Abbildung 6.1: Latenzmetriken je Modell und Szenario (Fehlerbalken: Standardabweichung aus vier Wiederholungen; Ladezeit ohne Fehlerbalken, da je Konfiguration einmalig gemessen).

Die TTFT kann im Allgemeinen von der Länge der Eingabe abhängen. Wie schon in Abschnitt 4.4.2 beschrieben, kann sich die Anzahl der Eingabe-Tokens trotz gleicher Promptlänge je Modell aufgrund der verschiedenen verwendeten Tokenizer unterscheiden. Tabelle 6.2 stellt diese Werte dar.

Tabelle 6.2: Anzahl der Eingabe-Tokens je Modell und Szenario. Die Werte stammen aus dem Tokenizer des Modells und fallen trotz gleichem Prompt unterschiedlich aus.

Modell	**Report**	**Artefakt**	**Vergleich**
Gemma 3 1B	417	446	740
Qwen 2.5 1.5B	439	482	770
Gemma 2 2B	415	444	736

Die Eingabe-Tokens fallen im Vergleichsszenario über alle Modelle hinweg am größten aus. Zudem zerlegt Qwen den Prompt über alle Szenarien hinweg in die meisten Tokens.

6.3 Ressourcenverbrauch

Darauffolgend werden die Ergebnisse der Messungen zum Ressourcenverbrauch vorgestellt.

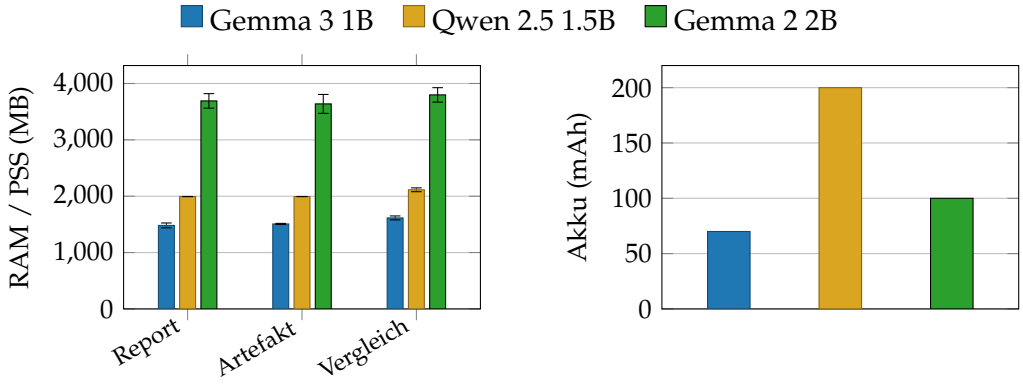
Tabelle 6.3: Ressourcenverbrauch je Modell (RAM als Mittel $\pm$ Standardabweichung).

Modell	**Szenario**	**RAM / PSS (MB)**	**Akku (mAh)***
Gemma 3 1B	Report	1482 ± 43	~70
	Artefakt	1508 ± 10	
	Vergleich	1615 ± 35	
Qwen 2.5 1.5B	Report	1994 ± 2	~200
	Artefakt	1994 ± 1	
	Vergleich	2114 ± 34	
Gemma 2 2B	Report	3691 ± 129	~100
	Artefakt	3638 ± 168	
	Vergleich	3798 ± 128	

*Verbrauch über den gesamten Messblock (15 Läufe) des jeweiligen Modells.

Tabelle 6.3 zeigt, dass der **RAM (PSS)** eine klare Rangfolge zwischen den Modellen besitzt. Während Gemma 3 1B mit (1482 MB) den geringsten Wert aufweist, liegt Qwen in der Mitte (ca. 2000 MB) und Gemma 2 2B am höchsten (3798 MB). Während das 1B-Modell von Gemma mit (ca. 70 mAh) den geringsten **Akkuverbrauch** aufweist, liegt der Verbrauch von Qwen 2.5 1.5B (ca. 200 mAh) und Gemma 2 2B (ca. 100 mAh) deutlich höher. Auffällig ist, dass das Modell Qwen am meisten Akku verbraucht. Über die gesamte Messreihe fällt die Restladung von rund 2234 mAh auf 1830 mAh, was einem Gesamtverbrauch von etwa 404 mAh in gut 16 Minuten entspricht (Tabelle B.1).

Der Ressourcenverbrauch ist in Abbildung 6.2 zusätzlich grafisch dargestellt.



(a) RAM / PSS (MB)

(b) Akku (mAh)

Abbildung 6.2: Ressourcenverbrauch je Modell (RAM/PSS mit Fehlerbalken als Standardabweichung; Akku als Näherungswert über den gesamten Messblock von 15 Läufen, daher ein Wert je Modell).

6.4 Stabilität

Alle 45 Messungen wurden mit dem Status „ok“ protokolliert. Das bedeutet, dass in keinem der Läufe eine Ausnahme auftrat, die der Benchmark abgefangen und in der CSV als Fehler vermerkt hätte (Abschnitt 5.6). Zusätzlich fehlt keine der 45 Zeilen in der Datei, was bei einem vorzeitigen Abbruch eines Durchlaufs der Fall wäre. Ein Absturz der Anwendung, eine Meldung über fehlenden Arbeitsspeicher oder ein Einfrieren der Oberfläche traten während der Messungen nicht auf. Das größere Modell Gemma 2 2B läuft ebenso stabil, obwohl es mit ca. 3798 MB den meisten RAM verbraucht.

6.5 Ausgabequalität

Die Ausgaben werden anhand der in Abschnitt 4.4.1 definierten Kriterien bewertet. Dies geschieht anhand mehrerer Beispiele. Da alle drei Modelle denselben Prompt (Abschnitt 5.4) verwenden, ist es möglich, die Qualität der Ausgaben direkt miteinander zu vergleichen.

Für die Bewertung wird eine artefaktbehaftete Session verwendet, die eine Delta-Dominanz aufweist. Der Fokus liegt auf dem Report-Szenario für alle drei Modelle, bei dem sie im Detail betrachtet werden. Anschließend werden die anspruchsvolleren Szenarien Artefakt-Erklärung und Vergleich zusammengefasst. Dadurch ist erkennbar, wie die Modelle mit steigenden Anforderungen umgehen.

Die an die Modelle übergebenen Feature-Texte beider Sessions sind in Abbildung A.1 und Abbildung A.2 im Anhang abgedruckt und ermöglichen den Abgleich der Ausgaben mit der tatsächlichen Eingabe.

> Das dominante Frequenzband war Delta, dessen Anteil in allen Abschnitten zwischen 56 % und 86 % lag. Die Signalqualität der Messungen war in allen Abschnitten gut. Auffällig waren Ausschlagsignale in den Abschnitten 1 und 4, wobei Kanal 4 und Kanal 2 jeweils die höchsten Werte erreichte.

Abbildung 6.3: Report-Szenario: unveränderte Ausgabe von Gemma 2 2B. Genannt werden das dominante Band, die Signalqualität und die auffälligen Kanäle.

Die Ausgabe von Gemma 2 2B (Abbildung 6.3) erkennt die Delta-Dominanz korrekt, bewertet die Signalqualität zutreffend und nennt die auffälligen Kanäle. Sie ist knapp und hält das Format ein. Alle vier Kriterien sind erfüllt.

> Die EEG-Session verzeichnete eine Konzentration von Delta-Zuständen (86 %), Theta-Zuständen (16 %), Alpha-Zuständen (13 %), Beta-Zuständen (6 %) und Gamma-Zuständen (5 %). Der größte Ausschlag wurde am Kanal 2 (3309 µV) festgestellt. Der Schwerpunkt lag hauptsächlich in den Sekundenbereichen 0–30, 30–60 und 60–91 Sekunden, wobei Delta- und Theta-Zustände am stärksten ausgeprägt waren. Es gab einige ungültige Messwerte, darunter ein signifikanter Ausschlag am Kanal 2.

Abbildung 6.4: Report-Szenario: unveränderte Ausgabe von Gemma 3 1B. Die aufgeführten Bandanteile summieren sich auf über 100 %.

Bei der Ausgabe von Gemma 3 1B (Abbildung 6.4) gibt es trotz übereinstimmender Delta-Dominanz einen logischen Fehler. Die erwähnten Bandanteile summieren sich auf über 100 %, da Werte verschiedener Segmente vermischt werden. Außerdem wird behauptet, dass es einen Anteil ungültiger Werte gibt, obwohl keine vorliegen. Korrektheit und Halluzinationsfreiheit sind daher nur eingeschränkt erfüllt.

> ### Bericht
> Die EEG-Messungen umfassen die Zeitraum von 0.0 s bis 121.432 s und wurden auf 8 KanÃ¤len gezählt. Die Signalqualität ist gut mit einer overall Signalqualität von 86 %. [. . .] Die dominante Frequenzbänder sind Delta (16 %), Theta (33 %), Alpha (60 %), Beta (15 %) und Gamma (13 %), wobei Delta und Theta die hauptsächlichste Frequenz bilden. [. . .]

Abbildung 6.5: Report-Szenario: unveränderte, gekürzte Ausgabe von Qwen 2.5 1.5B. Die Ausgabe enthält Formatierungszeichen und fehlerhaft kodierte Umlaute.

Bei der Ausgabe von Qwen 2.5 1.5B (Abbildung 6.5) treten mehrere Fehler auf. Vor allem die Formatvorgabe wird deutlich verletzt. Zunächst beginnt die Ausgabe mit einer Markdown-Überschrift (###), darüber hinaus enthält die Ausgabe fehlerhaft kodierte Umlaute wie „KanÃ¤len“ und ist zu lang. Zudem sind die Bandanteile falsch (Alpha mit 60 % als stärkstes Band, Summe über 100 %) und die Signalqualität ist frei erfunden. Keines der vier Kriterien wird in diesem Fall zuverlässig erfüllt.

Diese Erkenntnisse lassen sich auch in den anspruchsvolleren Szenarien beobachten. Obwohl Gemma 2 2B in der Artefakt-Erklärung korrekte technische Ursachen wie Bewegung, Elektroden oder Verbindung benennt, wird am Ende eine falsche Schlussfolgerung gezogen („das hohe Delta-Niveau stellt sicher, dass die Daten präzise sind“). Bei Gemma 3 1B werden neue Fachbegriffe wie „Delta-/Theta-/Gamma-Zell-Schwellenwert“ erfunden und weiterhin ungültige Messwerte behauptet. Qwen 2.5 1.5B liefert eine sehr schwache

Ausgabe, bei der die Bandzuordnungen widersprüchlich sind und Begriffe wie „Ungültiger Messergegenstand“ erfunden werden. Zudem treten hier wieder Formatierungsfehler auf. Gemma 2 2B vergleicht im Vergleich-Szenario die zwei Sessions sauber, bezeichnet die Ausschläge jedoch fälschlicherweise als zu gering, obwohl Werte über 4000 µV auftreten. Gemma 3 1B widerspricht sich selbst, indem beiden Sessions zugleich der größere Ausschlag an Kanal 2 zugeschrieben wird. Qwen 2.5 1.5B schneidet erneut schlecht ab. Es halluziniert mit Begriffen wie „Abgassen“ statt Frequenzbänder und gibt Steuerzeichen wörtlich aus („\n\n“). Auch hier werden erneut Formatierungsfehler gemacht.

Über alle Szenarien hinweg nehmen die Fehler mit steigender Komplexität zu, und die Modelle trennen sich klar nach Fähigkeit.

6.6 Zusammenfassende Gegenüberstellung

In Abbildung 6.6 sind jeweils die Stärken und Schwächen jedes Modells im direkten Vergleich zusammengefasst:

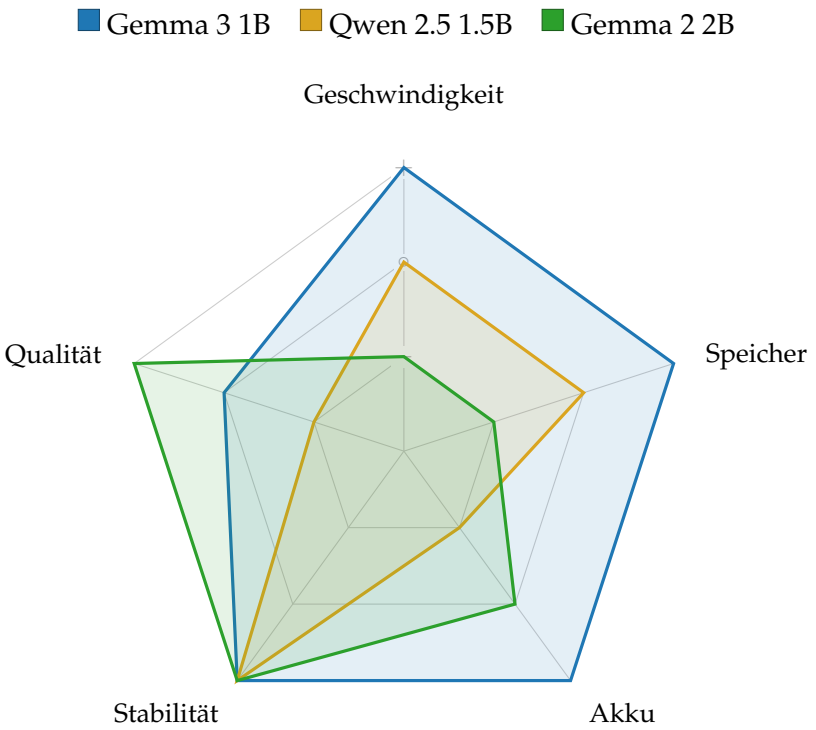


Abbildung 6.6: Netzdiagramm der zusammenfassenden Gegenüberstellung (weiter außen bedeutet besser).

Der Vergleich zeigt, dass kein Modell in allen Bereichen überlegen ist. Während Qwen 2.5 1.5B am schwächsten ausfällt, besitzen die Gemma-Modelle jeweils unterschiedliche Stärken. Gemma 3 1B bietet eine hohe Effizienz sowie wenig Speicher- und Akkuverbrauch, ist jedoch in den Punkten Qualität und Stabilität hinter Gemma 2 2B zurück. Letzteres glänzt mit einer hohen Ausgabequalität und Stabilität, ist jedoch langsamer und verbraucht mehr Ressourcen.

Interessant wird es bei der Frage, ob oder welches Modell sich für den praktischen Einsatz eignet und inwiefern, was nachfolgend in Kapitel 7 besprochen wird.

7 Diskussion

In diesem Kapitel werden die Ergebnisse der Evaluation eingeordnet und gedeutet. Anschließend werden die Einschränkungen der Arbeit diskutiert und Handlungsempfehlungen für den praktischen Einsatz lokaler Sprachmodelle in mobilen BCI-Anwendungen abgeleitet.

7.1 Einordnung der Ergebnisse

Die beiden Gemma-Modelle unterscheiden sich in ihren Stärken. Gemma 3 1B ist aufgrund seiner geringen Größe am effizientesten und hat den geringsten Speicher- und Akkubedarf sowie die höchste Geschwindigkeit. Die Parameterzahl beeinflusst dabei den Speicher- und Rechenaufwand je Token (Abschnitt 2.4.2). Innerhalb der Gemma-Familie zeigt sich das größere Gemma 2 2B als das Modell mit der höchsten Ausgabequalität.

Qwen 2.5 1.5B zeigt jedoch, dass die Größe allein die Qualität nicht bestimmt: Obwohl es sich von der Parameterzahl her in der Mitte der drei Modelle einordnet, liefert es die schlechtesten Ausgaben. Da alle Modelle unter denselben fairen Bedingungen (Kapitel 4) getestet wurden, ist dieser Unterschied allein den Fähigkeiten der Modelle zuzuschreiben. Qwen 2.5 1.5B hat die höchste Gesamtdauer und verbraucht am meisten Akku, obwohl es kleiner als Gemma 2 2B ist. Das Modell pendelt sich aber bei dem Rest in der Mitte ein. Der Grund für die lange Gesamtdauer liegt in der Länge der Antworten. Ein Zusammenhang ist zu erkennen: Je mehr Text ein Modell erzeugt, desto länger dauert die Berechnung und desto mehr Energie verbraucht sie. Qwen zeigt in den Messungen eine klar erhöhte Ausgabe an Tokens (vgl. Tabelle B.1). Diese Ausführlichkeit ist zugleich ein Qualitätsproblem, denn Qwen hält sich schlecht an die Vorgaben: Trotz der klaren Formatvorgaben im Prompt fällt die Antwort zu lang aus. Die Ausgabe ist so nicht nur inhaltlich am schwächsten, sondern kostet auch am meisten Zeit und Energie. Schlechte Qualität und hoher Verbrauch treten bei Qwen gemeinsam auf, möglicherweise durch das Nichteinhalten der Längenvorgaben. Ein fester Zusammenhang lässt sich jedoch aus den Messungen nicht ableiten.

Mögliche Gründe für das schlechte Einhalten der Vorgaben bei Qwen lassen sich aus den Messungen allein nicht klären. Das Kontextfenster ist ausgeschlossen, da im Report-Szenario mit jeweils 439 Eingabe- und 484 Ausgabe-Tokens ausreichend Spielraum bleibt (Tabelle B.1). Die Quantisierung spielt auch kaum eine Rolle, da alle Modelle mit 8 Bit ausgeführt werden. Wahrscheinlich liegt es an der Feinjustierung des Prompts, die speziell an das Modell Gemma 2 2B angepasst wurde (Abschnitt 7.2). Die Vorgaben im Prompt wirken bei einem Modell eines anderen Herstellers außerhalb der Gemma-Reihe möglicherweise anders und greifen schlechter. Denkbar ist zudem, dass sich die Modelle darin unterscheiden, wie strikt sie Längen- und Formatvorgaben umsetzen.

Ein weiterer Zusammenhang zeigt sich bei den Szenarien. Auffällig ist, dass die TTFT im Vergleichsszenario über alle Modelle hinweg deutlich länger dauert als bei Report und Artefakt-Erklärung. Dies lässt sich leicht erklären, da im Vergleich zwei Eingaben

verarbeitet werden, was zu einem längeren Prompt führt. Je mehr Text gelesen wird, desto länger dauert es, bis die Antwort generiert wird. Dies bestätigt sich in den Eingabe-Tokenzahlen (Tabelle 6.2), die im Vergleichsszenario am größten ausfallen. Dass Qwen den Prompt dabei in die meisten Tokens zerlegt, führt zu seiner erhöhten TTFT. Dieser Unterschied zwischen den Modellen in der Zerlegung kann durch das kleinere Vokabular (Abschnitt 4.4.2) entstehen, ebenso können die abweichenden Zerlegungsverfahren oder die Struktur des Prompts die Ursache sein. Welcher dieser Einflüsse überwiegt, bleibt offen, da es aus den Daten nicht sicher abgeleitet werden kann.

Die Ergebnisse zeigen somit, dass sich von der Modellgröße nicht unmittelbar auf die Ausgabequalität schließen lässt. Verschiedene Hersteller unterscheiden sich in Aufbau und Training der Modelle [10, 31], sodass die Größe als Anhaltspunkt nicht verlässlich genug ist. Die Stärken ergeben sich also aus dem Zusammenspiel von Größe und Fähigkeit. Bei der Stabilität unterscheiden sich die Modelle hingegen nicht. Alle Kombinationen laufen fehlerfrei durch. Nur Qwen 2.5 1.5B weist einen erhöhten Streuungswert im Vergleichsszenario auf, was sich durch seine lange Laufzeit unter Dauerlast begründen lässt. Am Ende ist für die eigentliche Fragestellung am entscheidendsten, ob die Modelle einwandfrei ihre Aufgabe erfüllt haben. Daher ist die Ausgabequalität das wichtigste Kriterium, vorausgesetzt, es gibt keine Probleme in der Technik selbst.

7.2 Einschränkungen

Während der gesamten Arbeit gibt es Einschränkungen, die die Ergebnisse beeinflussen können. Dazu gehören unter anderem Limitierungen durch Hardware, Designentscheidungen oder Entscheidungen bezogen auf die Messungen.

- **Begrenzte Testumgebung:** Die gesamte Arbeit basiert auf nur einem Testgerät, dem Samsung Galaxy S23. Es ist nicht sicher, wie die Evaluation auf anderen Hardwarekonfigurationen ausfallen würde.
- **Begrenzte Modellanzahl:** Aufgrund dessen, dass die gesamte Implementierung wie vorgegeben auf einem mobilen Gerät durchgeführt wird, sind die Möglichkeiten von lokalen Sprachmodellen begrenzt. Durch die selbst getroffene Wahl von MediaPipe als Laufzeitumgebung entfallen viele neuere Modelle, da die Schnittstelle nur eine begrenzte Auswahl unterstützt [15] oder die Modelle für mobile Geräte gar nicht optimiert sind [38]. Zudem wird strikt für einen fairen Vergleich darauf geachtet, dass alle mit einer Kontextgröße von 1280, einer 8-Bit-Quantisierung, einem gleichen Top-K-Wert mit 64 und dem .task-Format laufen, was zur Folge hat, dass noch mehr Modelle ausgeschlossen werden.
- **Keine systemintegrierten Modelle:** Aufgrund der begrenzten Unterstützung des Testgeräts ist es nicht möglich, systemintegrierte Modelle wie Gemini Nano einzubeziehen [18]. Zwar unterstützt das Testgerät Galaxy AI, dabei handelt es sich jedoch um vordefinierte Endnutzerfunktionen [33] und nicht um eine Schnittstelle, über die eigene Prompts an ein Sprachmodell übergeben werden könnten. Für den hier untersuchten Anwendungsfall ist es deshalb nicht einsetzbar.
- **Prompt auf Gemma 2 2B optimiert:** Die gesamte Promptfeinjustierung basiert auf dem Modell Gemma 2 2B, wodurch das Setup auf dieses Modell zugeschnitten ist. Davon betroffen ist sehr wahrscheinlich die Ausgabequalität sowie aufgrund der Länge der Ausgabe die Gesamtdauer und der Akkuverbrauch. Da die betroffenen

Werte als Ergebnis unter günstigen Bedingungen entstanden sind, stellt sich die Frage, ob Gemma 2 2B in den Vergleich gezogen werden soll. Da Ladezeit, TTFT, Durchsatz und Speicherbedarf davon unberührt bleiben, wird das Modell im Vergleich belassen. Die betroffenen Werte sind jedoch entsprechend einzuordnen.

- **Begrenzter Kontext:** Qwen hat im Vergleichsszenario 1244 Token verwendet und liegt damit knapp unter dem Gesamtbudget des Kontextfensters von 1280 Token. Dementsprechend bliebe weniger Platz für die Ausgabe, wenn der Prompt länger gestaltet wäre. Dies gilt für Alle Modelle, jedoch ist Qwen hier am stärksten betroffen.
- **Feste Messreihenfolge:** Die Reihenfolge der Messungen ist bewusst festgelegt (Abschnitt 4.4). Trotz einer Abkühlphase lässt sich nicht ausschließen, dass die zuletzt gemessenen Modelle durch das wärmere Gerät leicht benachteiligt werden (Abschnitt 2.4.2). Dasselbe gilt für den Ladezustand, der über die Messreihe hinweg sinkt. Der verbleibende Einfluss bleibt offen.
- **Ausgabesprache:** Alle Messungen erfolgen in deutscher Sprache, daher bleibt es offen, wie sich die Ausgabequalität und Tokenanzahl in englischer Sprache verhalten.

7.3 Handlungsempfehlungen

In diesem Abschnitt werden Handlungen auf Basis meiner Erkenntnisse empfohlen. Diese leiten sich aus den Ergebnissen der Evaluation ab. Die folgenden Empfehlungen gelten für den in dieser Arbeit untersuchten Rahmen, also für die beschriebene Implementierung mit der in Abschnitt 5.4 festgelegten Prompt-Struktur, für die drei Anwendungsszenarien aus Abschnitt 4.3 und für das Testgerät aus Abschnitt 4.4.2. Auf andere Geräte, Prompts oder Aufgaben lassen sie sich nicht ohne Weiteres übertragen.

7.3.1 Allgemeine Empfehlungen

Die Prompts sollten im besten Fall kurz und klar gehalten werden, damit sie in den Kontext passen. Das Modell sollte die Merkmale in einer klar verständlichen Form erhalten, ohne selbst überfordert zu sein, also keine Rohdaten, sondern aggregierte Merkmale. Darüber hinaus sollte man je nach Szenario das passende Modell auswählen, da die Ausgabequalität je nach Komplexität der Aufgabe variiert. Für die Benutzerfreundlichkeit ist es auch sinnvoll, eine wie hier verwendete Streaming-Mechanik einzubinden, um die gefühlte Wartezeit zu verringern. Dabei sollte bei der Ausgabe das Format nach Bedarf angepasst werden. Bei Qwen 2.5 1.5B ist es beispielsweise sinnvoll, die Antwort zu kürzen und formatierungstechnisch zu vereinfachen. Es ist dringend empfehlenswert, sich in medizinisch wichtigen Bereichen nicht allein auf solche Modelle zu verlassen. Ihre Ausgaben sollten immer mit Vorsicht und Skepsis betrachtet werden. Selbst das stärkste Modell in dieser Arbeit ist nicht fehlerfrei.

7.3.2 Modellempfehlung

Wenn der Fokus auf der Qualität liegen soll, geht die Empfehlung an das Modell Gemma 2 2B. Es hält sich am besten an die Vorgaben und bietet die beste Ausgabequalität. Dafür verbraucht es am meisten Speicher und ist am langsamsten, mit einer geringen Token-Rate und der längsten TTFT. Aufgrund des Speicherbedarfs von etwa 3,7 GB für den gesamten Prozess der Anwendung empfiehlt es sich, ein Testgerät mit weitaus mehr RAM zu nutzen. Mit Betriebssystem, einem Puffer und möglicherweise weiteren Anwendungen kann der Speicherbedarf schnell ansteigen.

Wer hingegen auf Geschwindigkeit, Speicher- und Akkuverbrauch setzen möchte, ist mit dem kleinsten Modell Gemma 3 1B am besten aufgehoben. Vor allem für schwache Geräte ist es die beste Wahl, da es die geringsten Anforderungen an das Gerät stellt.

Qwen 2.5 1.5B ist in diesem Rahmen nicht zu empfehlen. Es schneidet in puncto Qualität und Akkuverbrauch trotz seiner Größe am schlechtesten ab. Die Geschwindigkeit und der Speicherbedarf pendeln sich, mit Ausnahme der Gesamtdauer, wie erwartet in der Mitte ein, können jedoch nicht die mangelnde Ausgabequalität rechtfertigen.

Je nach Einsatzzweck haben die Modelle von Google einen klaren Vorteil. Wenn schnelle Antworten mit sehr geringer TTFT und einer niedrigen Gesamtdauer gefordert sind, führt kein Weg an kleinen Modellen wie Gemma 3 1B vorbei. Wenn hingegen Qualität wichtig und Zeit gegeben ist, beispielsweise als Hintergrundprozess ohne nötige sofortige Rückmeldung, ist Gemma 2 2B die beste Wahl.

Zusammenfassend lässt sich sagen, dass eine größere Parameteranzahl nicht automatisch eine bessere Wahl darstellt.

8 Fazit und Ausblick

Abschließend werden die Ergebnisse dieser Arbeit zusammengefasst. Darüber hinaus wird die Forschungsfrage zur Eignung lokaler Sprachmodelle für mobile BCI-Anwendungen beantwortet und daraufhin ein Ausblick auf mögliche Weiterentwicklungen oder bessere Ansätze in zukünftigen Arbeiten gegeben.

8.1 Fazit

Die bestehende EEG-Anwendung wurde um neue Funktionalitäten erweitert, darunter die Merkmalsextraktion, die Anbindung lokaler Sprachmodelle, ein Benchmark und neue Benutzeroberflächen.

Die Forschungsfrage, inwieweit sich lokal auf Android-Geräten ausgeführte Sprachmodelle für die Verarbeitung von EEG-Daten im Rahmen mobiler Brain-Computer-Interface-Anwendungen eignen, kann abschließend mit einem klaren Ja beantwortet werden. Die Evaluation hat eine stabile Verarbeitung der EEG-Daten durch alle drei getesteten Modelle gezeigt. Auch wenn Qwen 2.5 1.5B nicht überzeugen konnte, haben sich vor allem Gemma 2 2B und Gemma 3 1B als gut geeignet für die Verarbeitung von EEG-Daten erwiesen.

Die Modellgröße bestimmt zwar TTFT, Geschwindigkeit und Speicher, aber nicht automatisch Qualität oder Gesamtverbrauch. Da diese Arbeit gezeigt hat, dass kein Modell in allen Bereichen überlegen ist, ist es entscheidend, den richtigen Einsatzzweck zu definieren und das Modell danach auszuwählen.

Die im Folgenden genannten Empfehlungen sind im Rahmen dieser Implementierung, mit der genannten Struktur des Prompts und im Gebiet, bei dem es um die Auswertung von EEG-Sessions auf dem verwendeten Testgerät geht, zu verstehen. Es hängt vom jeweiligen Anwendungsfall ab, welches Modell am besten geeignet ist. Das 2B-Modell von Gemma ist die beste Wahl, wenn Zeit und Ressourcen keine Rolle spielen und die Qualität im Vordergrund stehen soll. Es kann unter anderem als Hintergrundprozess verwendet werden. Das 1B-Modell von Gemma ist für schnelle Antworten auf ressourcenärmeren Geräten die beste Wahl. Es empfiehlt sich für alle Fälle, bei denen der Nutzer eine sofortige Rückmeldung benötigt.

8.2 Ausblick

Die Grundlage dieser Arbeit öffnet die Tür für weitere Entwicklungen oder verbesserte Ansätze:

- **Mehr Testgeräte:** Da die Geräteauswahl auf ein Samsung Galaxy S23 beschränkt war, ist es nicht sicher, wie andere Chipsätze oder Hersteller die Ergebnisse beeinflussen würden. Man könnte die Evaluation auf weitere Geräte ausweiten und neue

Vergleiche anstellen, wie sich unter anderem schwächere gegen stärkere Hardware verhält.

- **Systemintegrierte Modelle:** Zudem ist es interessant zu wissen, wie systemintegrierte Modelle abschneiden würden. Zwar konnten sie in dieser Arbeit nicht verwendet werden, jedoch bieten sie, falls sie verfügbar sind, eine interessante Alternative zu frei deploybaren Modellen.
- **Detailreichere Sessions:** Zwar wurde darauf geachtet, dass die verwendeten Sessions fehlerfrei aufgenommen wurden, jedoch nicht darauf, in welcher Situation bzw. welchem mentalen Zustand sie getragen wurden. Man könnte sie beispielsweise beim Schlafen oder Konzentrieren tragen, um zu sehen, wie die KI mit diesen Daten umgeht.
- **Andere Segmentanzahl und Fenstergröße:** Da eine feste Segmentanzahl von vier und eine Fenstergröße von 256 verwendet wurden (Abschnitt 4.1.1), ist es offen, wie es mit anderen Parameterzahlen aussehen würde. Es wäre wissenswert, wie solche Änderungen die extrahierten Merkmale beeinflussen können.
- **Neuere Modelle & Runtimes:** Die Modellauswahl war auf das Testgerät beschränkt. Zudem wurde darauf geachtet, dass alle auf derselben Laufzeitumgebung mit jeweils gleichen Parametern laufen. Interessant wäre es zu evaluieren, wie sich andere aktuelle oder kommende Modelle mit anderen Laufzeitumgebungen verhalten. MediaPipe befindet sich mittlerweile im Wartungsmodus, weswegen dieser Punkt besonders relevant ist [15].
- **BCI-Daten für echte Steuerungsbefehle:** Die verarbeiteten Daten wurden nur als Eingabe an die KI-Modelle übergeben und nicht weiterverwendet. Man könnte diese Daten auch für Steuerungsbefehle verwenden, zum Beispiel in Echtzeit ohne Muskelkraft.
- **Zufällige Messreihenfolge:** Da Thermal Throttling die zuletzt gemessenen Modelle benachteiligen kann, wäre es interessant, die Reihenfolge zufällig zu wählen, um eine solche Verzerrung zu vermeiden.

Lokale Sprachmodelle bieten im mobilen BCI-Bereich Ansatzpunkte für weiterführende Arbeiten. Die Entwicklung neuer Modelle und Geräte schreitet kontinuierlich voran und bietet in Zukunft neue, größere Möglichkeiten, um vollständig offline und datenschutzfreundlich auf mobilen Geräten ohne jegliches Vorwissen zu arbeiten.

Literatur

[1] Nisreen Said Amer und Samir Brahim Belhaouari. „EEG Signal Processing for Medical Diagnosis, Healthcare, and Monitoring: A Comprehensive Review“. In: *IEEE Access* 11 (2023), S. 143116–143142. DOI: 10.1109/ACCESS.2023.3341419.

[2] Android Headlines. *One UI 8.5 Update Reaches Galaxy S23 Series Globally*. 2026. URL: https://www.androidheadlines.com/2026/05/one-ui-8-5-update-reaches-galaxy-s23-series-globally-whats-new.html (besucht am 19. 07. 2026).

[3] Naseem Babu, Jimson Mathew und A. P. Vinod. *Large Language Models for EEG: A Comprehensive Survey and Taxonomy*. Techn. Ber. arXiv, 2025. URL: https://arxiv.org/abs/2506.06353.

[4] Sarah Blum, Stefan Debener, Reiner Emkes, Nils Volkening, Sebastian Fudickar und Martin G. Bleichner. „EEG Recording and Online Signal Processing on Android: A Multiapp Framework for Brain-Computer Interfaces on Smartphone“. In: *BioMed Research International* 2017 (2017), S. 3072870. DOI: 10.1155/2017/3072870.

[5] Hongyu Chen, Weiming Zeng, Chengcheng Chen u. a. „EEG Emotion Copilot: Optimizing lightweight LLMs for emotional EEG interpretation with assisted medical record generation“. In: *Neural Networks* 192 (2025), S. 107848. DOI: 10.1016/j.neunet.2025.107848.

[6] DeepSeek-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. Techn. Ber. DeepSeek, 2025. URL: https://arxiv.org/abs/2501.12948.

[7] g.tec medical engineering. *Unicorn Bluetooth Protocol*. URL: https://github.com/unicorn-bi/Unicorn-Suite-Hybrid-Black/blob/master/Unicorn%20Bluetooth%20Protocol/UnicornBluetoothProtocol.pdf (besucht am 02. 07. 2026).

[8] g.tec medical engineering. *Unicorn Hybrid Black – User Manual*. URL: https://github.com/unicorn-bi/Unicorn-Suite-Hybrid-Black-User-Manual/blob/main/UnicornHybridBlack.md (besucht am 02. 07. 2026).

[9] g.tec medical engineering. *Unicorn Hybrid Black BCI Platform*. URL: https://www.gtec.at/product/unicorn-hybrid-black-bci-platform/ (besucht am 02. 07. 2026).

[10] Gemma Team. *Gemma 2: Improving Open Language Models at a Practical Size*. Techn. Ber. Google DeepMind, 2024. URL: https://arxiv.org/abs/2408.00118.

[11] Gemma Team. *Gemma 3 Technical Report*. Techn. Ber. Google DeepMind, 2025. URL: https://arxiv.org/abs/2503.19786.

[12] Gemma Team. *Gemma 4 – Model Card*. Techn. Ber. Google DeepMind, 2026. URL: https://ai.google.dev/gemma/docs/core/model_card_4?hl=de.

[13] Google. *BatteryManager – Android API reference*. 2024. URL: https://developer.android.com/reference/android/os/BatteryManager (besucht am 12. 07. 2026).

[14] Google. *Gemini Nano*. 2024. URL: https://developer.android.com/ai/gemini-nano (besucht am 24. 06. 2026).

[15] Google. *LLM Inference Guide – MediaPipe*. 2024. URL: https://developers.google.com/edge/mediapipe/solutions/genai/llm_inference (besucht am 24. 06. 2026).

[16] Google. *Memory allocation among processes*. 2024. URL: https://developer.android.com/topic/performance/memory-management (besucht am 12. 07. 2026).

[17] Google. *OutOfMemoryError – Android API reference*. 2024. URL: https://developer.android.com/reference/java/lang/OutOfMemoryError (besucht am 20. 07. 2026).

[18] Google. *Overview of the ML Kit GenAI APIs*. 2025. URL: https://developers.google.com/ml-kit/genai (besucht am 09. 07. 2026).

[19] Qingzhe Guo, Tu Ouyang und An Wang. „Large Language Models on Mobile Devices: A Measurement Study of Single- and Multi-Instance Execution“. In: *Proceedings of the 2nd International Workshop on Edge and Mobile Foundation Models (EdgeFM '25)*. Hong Kong, China: ACM, 2025, S. 43–48. DOI: 10.1145/3737902.3768359.

[20] D. Hazarika, S. Chhaba, R. Ransing und C. N. Gupta. „On-device single-channel EEG classification on Android smartphones using lightweight machine learning models“. In: *Biomedical Physics & Engineering Express* (2026). DOI: 10.1088/2057-1976/ae3ca4.

[21] IBM. *Was ist ein Kontextfenster?* 2024. URL: https://www.ibm.com/de-de/think/topics/context-window (besucht am 12. 07. 2026).

[22] Mainak Jas, Denis A. Engemann, Yousra Bekhti, Federico Raimondo und Alexandre Gramfort. „Autoreject: Automated artifact rejection for MEG and EEG data“. In: *NeuroImage* 159 (2017), S. 417–429. DOI: 10.1016/j.neuroimage.2017.06.030.

[23] Dah-Jing Jwo, Wei-Yeh Chang und I-Hua Wu. „Windowing Techniques, the Welch Method for Improvement of Power Spectrum Estimation“. In: *Computers, Materials & Continua* 67.3 (2021), S. 3983–4003. DOI: 10.32604/cmc.2021.014752.

[24] Thomas Kuttner. *Praxiswissen Schwingungsmesstechnik*. 1. Aufl. Wiesbaden: Springer Vieweg, 2015. ISBN: 978-3-658-04637-8. DOI: 10.1007/978-3-658-04638-5.

[25] LiteRT Community. *Gemma2-2B-IT (LiteRT)*. 2025. URL: https://huggingface.co/litert-community/Gemma2-2B-IT (besucht am 22. 08. 2026).

[26] LiteRT Community. *Gemma3-1B-IT (LiteRT)*. 2025. URL: https://huggingface.co/litert-community/Gemma3-1B-IT (besucht am 22. 08. 2026).

[27] LiteRT Community. *Qwen2.5-1.5B-Instruct (LiteRT)*. 2025. URL: https://huggingface.co/litert-community/Qwen2.5-1.5B-Instruct (besucht am 16. 07. 2026).

[28] Llama Team. *The Llama 3 Herd of Models*. Techn. Ber. Meta AI, 2024. URL: https://arxiv.org/abs/2407.21783.

[29] Akshaj Murhekar, Christina Liu, Abhijit Mishra, Shounak Roychowdhury und Jacek Gwizdka. „SENSE: Efficient EEG-to-Text via Privacy-Preserving Semantic Retrieval“. In: *Proceedings of the ACM International Conference on Multimodal Interaction (ICMI)*. 2026. URL: https://arxiv.org/abs/2603.17109.

[30] Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr und Adel Bibi. „Language Model Tokenizers Introduce Unfairness Between Languages“. In: *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. 2023. URL: https://proceedings.neurips.cc/paper_files/paper/2023/file/74bb24dca8334adce292883b4b651eda-Paper-Conference.pdf.

[31] Qwen Team. *Qwen2.5 Technical Report*. Techn. Ber. Alibaba Group, 2024. URL: https://arxiv.org/abs/2412.15115.

[32] Qwen Team. *Qwen3 Technical Report*. Techn. Ber. Alibaba Group, 2025. URL: https://arxiv.org/abs/2505.09388.

[33] Samsung Electronics. *Galaxy AI – Funktionen*. 2024. URL: https://www.samsung.com/de/support/mobile-devices/galaxy-ai-funktionen/ (besucht am 12. 08. 2026).

[34] Samsung Electronics. *Galaxy S23 – Technische Daten*. 2023. URL: https://www.samsungmobilepress.com/media-assets/galaxy-s23?tab=specs (besucht am 19. 07. 2026).

[35] Steven W. Smith. *The Scientist and Engineer's Guide to Digital Signal Processing*. San Diego, CA: California Technical Publishing, 1997. ISBN: 978-0966017632. URL: https://www.dspguide.com/.

[36] Ashish Vaswani, Noam Shazeer, Niki Parmar u. a. „Attention Is All You Need". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017.

[37] Piotr Wendykier. *JTransforms*. URL: https://github.com/wendykierp/jtransforms (besucht am 02. 07. 2026).

[38] Yue Zheng, Yuhao Chen, Bin Qian, Xiufang Shi, Yuanchao Shu und Jiming Chen. „A Review on Edge Large Language Models: Design, Execution, and Applications". In: *ACM Computing Surveys* 57.8 (2025). DOI: 10.1145/3719664.

[39] Yang Zhou, Feng Liang, Ting-wu Chin und Diana Marculescu. „Play It Cool: Dynamic Shifting Prevents Thermal Throttling". In: *DyNN Workshop at the 39th International Conference on Machine Learning (ICML)*. Baltimore, MD, USA, 2022. URL: https://arxiv.org/abs/2206.10849.

Abbildungsverzeichnis

2.1 Zeit- und Frequenzbereich 6

3.1 Verwendete Sprachmodelle 12
3.2 Komponenten des verwendeten EEG-Geräts g.tec Unicorn Hybrid Black (eigene Beschriftung, erstellt mit draw.io) [9]. 14

4.1 Datenfluss der Merkmalsextraktion 18
4.2 Segmentierung einer Session 19
4.3 Zerlegung eines Segments in überlappende, mit einer Hann-Funktion gewichtete Analysefenster (50 % Überlappung) für die Welch-Methode (erstellt mit draw.io). 20

5.1 Architektur der Anwendung 30
5.2 Sequenzdiagramm der Analyse 32
5.3 Beispiel eines Feature-Texts 38
5.4 Session-Detailansicht 43
5.5 Analyse-Dialog 44
5.6 Ergebnis-Ansicht 45
5.7 Benchmark-Dialog 47
5.8 Sequenzdiagramm des Benchmarks 48

6.1 Latenzmetriken je Modell und Szenario (Fehlerbalken: Standardabweichung aus vier Wiederholungen; Ladezeit ohne Fehlerbalken, da je Konfiguration einmalig gemessen). 55
6.2 Ressourcenverbrauch je Modell (RAM/PSS mit Fehlerbalken als Standardabweichung; Akku als Näherungswert über den gesamten Messblock von 15 Läufen, daher ein Wert je Modell). 57
6.3 Ausgabe von Gemma 2 2B 58
6.4 Ausgabe von Gemma 3 1B 58
6.5 Ausgabe von Qwen 2.5 1.5B 58
6.6 Netzdiagramm der zusammenfassenden Gegenüberstellung (weiter außen bedeutet besser). 59

A.1 Feature-Text der Session A 77
A.2 Feature-Text der Session B 77

Tabellenverzeichnis

2.1 EEG-Frequenzbänder 6

3.1 Aufbau eines Samples 14

3.2 Einordnung gegenüber verwandten Arbeiten 15

4.1 Übersicht der Metriken 25

6.1 Latenzmetriken je Modell und Szenario (Mittel aus vier Wiederholungen ± Standardabweichung). 54
6.2 Eingabe-Tokens 56
6.3 Ressourcenverbrauch je Modell (RAM als Mittel ± Standardabweichung). 56

B.1 Rohdaten der Messungen 79

Listings

5.1 Zerlegung in Segmente 33
5.2 Band-Power je Kanal 34
5.3 Leistungsspektrum eines Fensters 35
5.4 Zuordnung zu den Bändern 36
5.5 Relative Band-Power 37
5.6 Laden des Modells 39
5.7 Streamende Ausgabe 40
5.8 Prompts je Szenario 41
5.9 Live-Anzeige der Ausgabe 44
5.10 Ablauf des Benchmarks 49
5.11 Wiederholte Messung 50

Abkürzungsverzeichnis

API Programmierschnittstelle
BBPE Byte-Level Byte-Pair Encoding
BCI Brain-Computer-Interface
CPU Hauptprozessor
CSV Comma-Separated Values
EEG Elektroenzephalographie
FFT Fast Fourier Transform
GPU Grafikprozessor
ID Identifikationsnummer
LLM Large Language Model
OOM Out of Memory
PSS Proportional Set Size
RAM Arbeitsspeicher
SNR Signal-Rausch-Verhältnis
TTFT Zeit bis zum ersten Token

Anhang

A Aufbereitete Feature-Texte der Evaluation

Die folgenden Feature-Texte (Abbildung A.1, Abbildung A.2) wurden vom `FeatureFormatter` aus den beiden Sessions erzeugt und den Modellen als Eingabe übergeben (vgl. Abschnitt 6.5). Sie ermöglichen es, die Modellausgaben gegen die tatsächliche Eingabe zu prüfen.

```
EEG-Session, Dauer 121.432s, 8 Kanäle.

Abschnitt 1 (0.0–30.352s):
    Delta 86%, Theta 4%, Alpha 5%, Beta 3%, Gamma 2%
    Ungültige Messwerte: 0.0% ungültig, größter Ausschlag Kanal 4 (4419 µV)

Abschnitt 2 (30.356–60.708s):
    Delta 59%, Theta 16%, Alpha 13%, Beta 7%, Gamma 5%
    Ungültige Messwerte: 0.0% ungültig, größter Ausschlag Kanal 2 (3309 µV)

Abschnitt 3 (60.712–91.064s):
    Delta 56%, Theta 18%, Alpha 13%, Beta 7%, Gamma 6%
    Ungültige Messwerte: 0.0% ungültig, größter Ausschlag Kanal 2 (2874 µV)

Abschnitt 4 (91.068–121.432s):
    Delta 63%, Theta 11%, Alpha 11%, Beta 8%, Gamma 8%
    Ungültige Messwerte: 0.0% ungültig, größter Ausschlag Kanal 2 (2948 µV)
```

Abbildung A.1: Feature-Text der Session A, wie er den Modellen übergeben wurde. Er ist die Grundlage für das Report- und das Artefakt-Szenario.

```
EEG-Session, Dauer 122.128s, 8 Kanäle.

Abschnitt 1 (0.0–30.528s):
    Delta 86%, Theta 5%, Alpha 3%, Beta 4%, Gamma 2%
    Ungültige Messwerte: 0.0% ungültig, größter Ausschlag Kanal 2 (5230 µV)

Abschnitt 2 (30.532–61.06s):
    Delta 77%, Theta 9%, Alpha 6%, Beta 5%, Gamma 3%
    Ungültige Messwerte: 0.0% ungültig, größter Ausschlag Kanal 2 (4772 µV)

Abschnitt 3 (61.064–91.592s):
    Delta 73%, Theta 10%, Alpha 6%, Beta 7%, Gamma 4%
    Ungültige Messwerte: 0.0% ungültig, größter Ausschlag Kanal 2 (3878 µV)

Abschnitt 4 (91.596–122.128s):
    Delta 66%, Theta 15%, Alpha 8%, Beta 6%, Gamma 5%
    Ungültige Messwerte: 0.0% ungültig, größter Ausschlag Kanal 2 (3518 µV)
```

Abbildung A.2: Feature-Text der Session B, wie er den Modellen übergeben wurde. Er wird nur im Vergleichsszenario als zweite Session genutzt.

B Rohdaten der Benchmark-Messungen

In Tabelle B.1 sind alle 45 Messungen vollständig aufgelistet. Diese dienen zur Nachvollziehbarkeit der Evaluation.

Tabelle B.1: Rohdaten aller Benchmark-Durchläufe. `Load` bezeichnet die Ladezeit, `Infer` die reine Inferenzdauer. Die im Haupttext genannte Gesamtdauer entspricht der Spalte `Infer`, die Ladezeit wird dort getrennt ausgewiesen.

Zeit	Modell	Szenario	Prompt (Tok.)	Ausgabe (Tok.)	Load (ms)	Infer (ms)	TTFT (ms)	Tok/s	PSS (MB)	Akku (µAh)	Akku (%)	Status
19:32:07	1b	report	417	145	447	13116	4120	11.06	1854	2234115	59	ok
19:32:14	1b	report	417	145	447	6906	2047	21.00	1489	2234115	59	ok
19:32:21	1b	report	417	145	447	6639	1919	21.84	1527	2234115	59	ok
19:32:27	1b	report	417	145	447	6597	1883	21.98	1424	2234115	59	ok
19:32:34	1b	report	417	145	447	6658	1908	21.78	1488	2234115	59	ok
19:32:47	1b	artefakt	446	290	339	11596	2013	25.01	1502	2211885	59	ok
19:32:58	1b	artefakt	446	290	339	11476	1974	25.27	1503	2211885	59	ok
19:33:10	1b	artefakt	446	290	339	11401	1910	25.44	1503	2197065	58	ok
19:33:21	1b	artefakt	446	290	339	11424	1985	25.39	1504	2197065	58	ok
19:33:33	1b	artefakt	446	290	339	11477	1912	25.27	1523	2197065	58	ok
19:33:45	1b	vergleich	740	117	289	10107	5890	11.58	1568	2178540	58	ok
19:33:54	1b	vergleich	740	117	289	9494	5444	12.32	1633	2178540	58	ok
19:34:05	1b	vergleich	740	117	289	10529	6554	11.11	1635	2178540	58	ok
19:34:15	1b	vergleich	740	117	289	9918	5852	11.80	1628	2163720	58	ok
19:34:24	1b	vergleich	740	117	289	9606	5582	12.18	1562	2163720	58	ok
19:34:42	2b	report	415	72	871	15173	6877	4.75	3500	2148900	57	ok
19:34:56	2b	report	415	72	871	14264	6133	5.05	3497	2148900	57	ok
19:35:10	2b	report	415	72	871	14095	5896	5.11	3757	2137785	57	ok
19:35:26	2b	report	415	72	871	14907	6747	4.83	3757	2137785	57	ok
19:35:40	2b	report	415	72	871	13735	5577	5.24	3753	2126670	57	ok
19:35:58	2b	artefakt	444	88	467	16864	6440	5.22	3494	2126670	57	ok
19:36:17	2b	artefakt	444	88	467	18489	6723	4.76	3494	2111850	56	ok
19:36:35	2b	artefakt	444	88	467	17780	6225	4.95	3494	2111850	56	ok
19:36:55	2b	artefakt	444	88	467	18987	7037	4.63	3755	2100735	56	ok
19:37:14	2b	artefakt	444	88	467	19005	7034	4.63	3810	2089620	56	ok
19:37:43	2b	vergleich	736	78	516	26207	16549	2.98	3606	2078505	55	ok
19:38:10	2b	vergleich	736	78	516	27347	16459	2.85	3605	2067390	55	ok
19:38:38	2b	vergleich	736	78	516	27050	16408	2.88	3864	2059980	55	ok
19:39:05	2b	vergleich	736	78	516	27143	16916	2.87	3861	2059980	55	ok
19:39:33	2b	vergleich	736	78	516	27229	16618	2.86	3861	2048865	55	ok
19:40:11	qwen	report	439	484	729	36294	6744	13.34	1992	2030340	54	ok

Fortsetzung auf der nächsten Seite

Zeit	Modell	Szenario	Prompt (Tok.)	Ausgabe (Tok.)	Load (ms)	Infer (ms)	TTFT (ms)	Tok/s	PSS (MB)	Akku (µAh)	Akku (%)	Status
19:40:46	qwen	report	439	484	729	34373	5242	14.08	1992	2015520	54	ok
19:41:20	qwen	report	439	484	729	34215	5335	14.15	1994	2004405	53	ok
19:41:55	qwen	report	439	484	729	34371	5389	14.08	1995	1993290	53	ok
19:42:29	qwen	report	439	484	729	34775	5487	13.92	1996	1978470	53	ok
19:42:58	qwen	artefakt	482	357	266	27101	5489	13.17	1993	1967355	52	ok
19:43:25	qwen	artefakt	482	357	266	26562	5265	13.44	1993	1952535	52	ok
19:43:52	qwen	artefakt	482	357	266	26847	5493	13.30	1995	1941420	52	ok
19:44:18	qwen	artefakt	482	357	266	26712	5253	13.36	1995	1926600	51	ok
19:44:45	qwen	artefakt	482	357	266	26670	5212	13.39	1994	1915485	51	ok
19:45:27	qwen	vergleich	770	474	272	39502	13959	12.00	2095	1904370	51	ok
19:46:06	qwen	vergleich	770	474	272	39482	13806	12.01	2096	1893255	50	ok
19:46:48	qwen	vergleich	770	474	272	41471	13328	11.43	2096	1863615	50	ok
19:47:32	qwen	vergleich	770	474	272	43520	14214	10.89	2098	1852500	49	ok
19:48:15	qwen	vergleich	770	474	272	43523	14613	10.89	2165	1830270	49	ok

Kolophon

Dieses Dokument wurde mit der LaTeX-Vorlage für Abschlussarbeiten an der htw saar im Bereich Informatik/Mechatronik-Sensortechnik erstellt (Version 2.25, August 2024). Die Vorlage wurde von Yves Hary und André Miede entwickelt (mit freundlicher Unterstützung von Thomas Kretschmer, Helmut G. Folz und Martina Lehser). Daten: (F)10.95 – (B)426.79135pt – (H)688.5567pt